Datawarehouse design

4

In het vorige hoofdstuk is uiteengezet waarom een datawarehouse (of data lake) nodig is voor een succesvolle implementatie van Business Intelligence. In dat hoofdstuk zijn veel problemen genoemd die kunnen optreden zonder datawarehouse. In alle gevallen is gezegd dat een centraal datawarehouse de (of een deel van de) oplossing is.
In dit hoofdstuk gaan we kijken naar het ontwerp van een datawarehouse. We zullen uitgebreid ingaan op dimensioneel modelleren. Soms wordt een volledig datawarehouse opgezet volgens de theorie van dimensioneel modelleren. Tegenwoordig zie je vaak Data Vault-datawarehouses of Data Lake-implementaties die gebruikmaken van dimensioneel gemodelleerde datamarts. In beide gevallen is dimensioneel modelleren belangrijk. Naast het dimensioneel modelleren komt de methode van Inmon kort aan bod, alsmede de Data Vault.

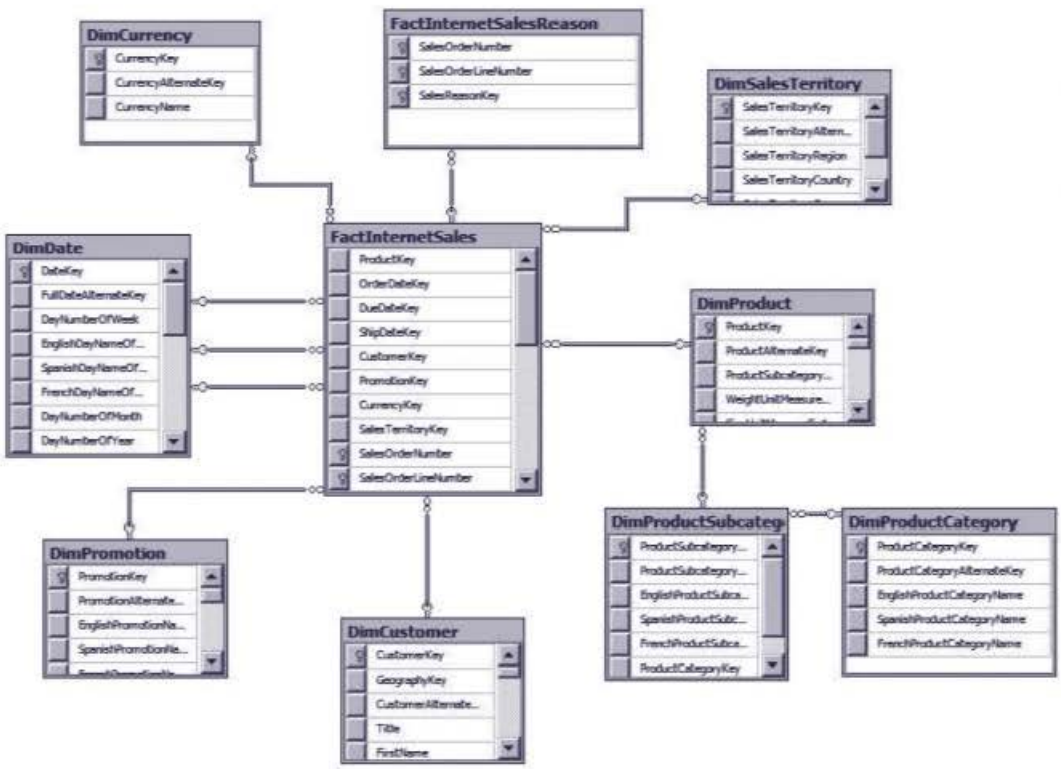
4.1 Dimensioneel modelleren

4.1.1 Inleiding

Het eerste argument dat in de eerste paragraaf van het vorige hoofdstuk werd genoemd om niet te rapporteren op operationele databases, was het feit dat deze over het algemeen genormaliseerd zijn. Normaliseren leidt tot veel verschillende tabellen. Dit maakt het schrijven van query's complex en het zorgt veelal voor slechte performance van query's. Dimensioneel modelleren is een manier van databaseontwerp, die poogt om deze tekortkomingen weg te nemen. Uitgangspunt is dat de database een voornamelijk alleen-lezen-workload (read-only) krijgt. Voor een database met een OLTP-workload is dimensioneel modelleren niet geschikt.
Dimensioneel modelleren leidt tot een database met een zogenaamd stermodel. In figuur 4.1 zie je een voorbeeld van een stermodel. Een database die bestaat uit slechts één stermodel bevat informatie over één proces binnen een organisatie. Een database die het bij één ster laat, wordt een datamart genoemd. In de theorie van Kimball is het datawarehouse de verzameling van alle stermodellen, die op hun beurt alle businessprocessen beschrijven.

Een **datamart** is een database waarvan de tabelstructuur een ster vormt en die één proces uit de organisatie beschrijft.

In de theorie van Kimball bestaat het datawarehouse uit de verzameling van alle stermodellen die tezamen de hele organisatie beschrijven.



Figuur 4.1 Voorbeeld van een stermodel

Een stermodel bestaat uit twee soorten tabellen:
- de feitentabel;
- dimensietabellen.

Het stermodel ontleent zijn naam aan het feit dat de feitentabel centraal staat en de dimensietabellen er in een soort sterstructuur omheen staan.

De feitentabel

Centraal in figuur 4.1 staat de tabel FactInternetSales. Dit is de feitentabel. In het voorbeeld zie je een stermodel dat een verkoopproces weerspiegelt. Als je aan een verkoopmanager vraagt hoe het gaat, zal hij dingen zeggen als: 'Het gaat goed, want de *omzet* is x miljoen.' 'Het gaat niet naar verwachting, want de *marge* is te laag.' 'Het *aantal verkochte* producten is weliswaar gestegen, maar de *kosten* die we daarvoor hebben moeten maken zijn harder gestegen.' Omzet,

marge, aantal stuks verkocht en kosten zijn allemaal meetbare grootheden die iets zeggen over het verkoopproces. Het zijn de getallen die het proces meetbaar hebben gemaakt. Of anders gezegd, het zijn de feiten. Deze feiten sla je op in de feitentabel, de tabel die centraal staat in het stermodel.



De **feitentabel** is de tabel waar grootheden die een proces meetbaar maken in worden opgeslagen. KPI's worden gebaseerd op deze grootheden.

Dimensietabellen

Feiten alleen zijn niet genoeg, getallen zeggen niets. Getallen zullen altijd in context geplaatst moeten worden. Dat de omzet één miljoen euro bedraagt, zegt eigenlijk nog niets. De volgende zin heeft wel betekenis: een opleider heeft één miljoen omzet gerealiseerd met het geven van Business Intelligence-cursussen in het afgelopen half jaar. De betekenis komt van de context. Opeens gaat het over een concreet product in een concrete periode. De context, de betekenis, komt van de dimensies. In het voorbeeld zie je onder andere een productdimensie (DimProduct) en een datumdimensie (DimDate) die de context van het voorbeeld bevatten.



Dimensietabellen zijn de tabellen die de context bevatten die betekenis geeft aan de feiten.

Het idee achter het stermodel is dat iemand die het proces kent, het stermodel snapt. Een stermodel is intuïtief. Iedereen op de afdeling verkoop weet dat er producten verkocht worden en welke dat zijn. Iedereen weet dat er targets (doelstellingen) zijn geformuleerd per periode. Met die kennis wordt het plaatje eenvoudig om te lezen. Daarmee wordt de database eenvoudig om te gebruiken. Met als bijkomend voordeel: als het voor mensen eenvoudiger wordt, geldt dat over het algemeen ook voor de computer, waardoor de performance verbetert. Tevens zijn self-service BI-tools beter in staat om de gebruiker te helpen, waardoor het nog eenvoudiger en dus laagdrempeliger wordt voor de mens.



Een **stermodel** is voor een vakinhoudelijk deskundige een gemakkelijk te begrijpen, en dus te gebruiken, model.

4.1.2 Modelleren

Ralph Kimball is de grondlegger van het dimensioneel modelleren. Een van de standaardwerken over dimensioneel modelleren is het boek van Ralph Kimball

en Margy Ross *The Data Warehouse Toolkit* (2013). In dit boek beschrijven zij vier hoofdstappen om tot een stermodel te komen. Deze stappen zijn:

1. Kies het te modelleren proces.
2. Bepaal het te gebruiken detailniveau.
3. Bepaal de van toepassing zijnde dimensies.
4. Bepaal de relevante feiten.

We zullen deze stappen een voor een kort bespreken.

Kies het te modelleren proces

De eerste stap in het komen tot een dimensioneel ontwerp, is het kiezen van het te modelleren proces. Hieruit volgt meteen een heel belangrijk kenmerk van het dimensioneel modelleren: wat je gaat modelleren is een proces. Je vangt een proces in een databasemodel. Het proces staat dus centraal. Niet voor niets is er in de eerste hoofdstukken op gehamerd dat Business Intelligence geen ICT-'ding' is, noch een technisch iets. Het gaat om de bedrijfsvoering, om de processen.

Vanuit de tot nu toe behandelde stof klinkt het bovenstaande hopelijk logisch. Vanuit een klassieke database-invalshoek is het echter niet vanzelfsprekend. Veel mensen leren tijdens hun opleiding dat je een database moet normaliseren, een term die in hoofdstuk 3 ook is gebruikt. Bij normaliseren kijk je voornamelijk naar de structuur van de gegevens. Je modelleert de gegevens en hun onderlinge verbanden, niet zozeer het proces. Een dimensioneel gemodelleerde database staat daarmee dichter bij de gebruikers, waardoor de database makkelijker te gebruiken wordt. Daarmee is al een belangrijke doelstelling behaald.



Bij **normaliseren** modelleer je de gegevens en hun verbanden. Bij *dimensioneel modelleren* modelleer je de bedrijfsprocessen. Dimensioneel modelleren staat dichter bij de eindgebruikers.

Het kiezen van het proces dat je gaat modelleren kan gemakkelijk zijn of juist niet. Soms is de vraag waaruit het project voortkomt expliciet verbonden aan een proces. Bijvoorbeeld: het verkoopproces moet geoptimaliseerd worden en daartoe hebben we meer en betere inzichten nodig in dat verkoopproces. Het kan ook zijn dat je uit de informatieanalyse heel veel verschillende soorten informatiebehoeften hebt gedestilleerd die allemaal te maken hebben met andere zaken binnen de organisatie. Het is dan van belang om te beseffen dat je voor elk apart proces een apart stermodel moet maken. De ster modelleert immers het proces. Maar waar moet je beginnen?



Figuur 4.2 Beslismatrix

Figuur 4.2 is een herhaling van figuur 2.8 die je gezien hebt bij het verhaal over scope in hoofdstuk 2.
Sommige processen zijn heel complex. Complexe processen met veel uitzonderingen zijn moeilijk te modelleren. Daarom neemt het risico op fouten in het model toe. Fouten in het model leiden tot fouten in de rapportages. Fouten in de rapportages leiden ertoe dat de gebruikers de datamart niet vertrouwen en dus niet gaan gebruiken. Project mislukt.
Er zijn ook relatief makkelijke processen. Gemakkelijk is subjectief en kan te maken hebben met de kennis en ervaring van de betrokken gebruiker en/of met de ervaring van degene die het model moet maken. Duidelijk is dat de kans op succes toeneemt, terwijl als extra bonus de benodigde tijd om de database te bouwen afneemt.



Start het bouwen van het datawarehouse met het modelleren van een proces dat zich in de rechterbovenhoek van het kwadrant van figuur 4.2 bevindt: gemakkelijk en met veel meerwaarde!

Bepaal het te gebruiken detailniveau

Nadat het te modelleren proces is gekozen, moet je nadenken over het gewenste detailniveau, ofwel de grain. Deze 'keuze' volgt uiteraard rechtstreeks uit de informatieanalyse. Toch is het wenselijk om hier goed over na te denken, voornamelijk vanuit een performance-overweging.



De **grain** is het detailniveau van de datamart.

Het detailniveau dat je kiest bepaalt hoe groot de database gaat worden. Het heeft daarmee ook rechtstreeks invloed op de performance die je mag verwachten. Aan de andere kant van de medaille bepaalt het welke vragen wel en niet beantwoord kunnen worden. Je kunt nooit gedetailleerdere vragen beantwoorden dan die van de grain van de datamart.



Voorbeeld 4.1

Stel dat een internetwinkel gemiddeld honderd orders per dag krijgt en dat er per order gemiddeld twee producten worden gekocht. Met een maand van dertig dagen levert dat 100 x 2 x 30 = 6.000 records op per maand. Met tien jaar aan historie in de database, zijn dat 720.000 records.
Als je als grain maandomzet kiest, dus één record per maand, houd je nog maar 120 records over.

Bovenstaand voorbeeld is wat overtrokken om het punt duidelijk te maken. Uiteraard verlies je heel veel informatie met maand als laagste detailniveau. In een maand worden veel verschillende producten verkocht. Je kunt niet meer analyseren welke producten hoeveel omzet hebben opgebracht. Hetzelfde geldt voor klanten. Hoeveel klanten hebben orders besteld en hoeveel besteedt een klant gemiddeld per maand? Dat zijn allemaal vragen die niet meer beantwoord kunnen worden. Financiële rapportages met omzet per maand, kwartaal of zelfs jaar zijn wel razend snel.



Een laag detailniveau resulteert in een grote database met bijbehorende performance-uitdagingen, maar levert wel de meeste mogelijkheden in termen van welke vragen beantwoord kunnen worden.

Met de huidige hardware en met de huidige verwachtingspatronen van gebruikers weegt het laatste argument vele malen zwaarder dan het eerste. Mensen gaan op zoek naar meer details. Als dat niet mogelijk blijkt, raken ze teleurgesteld in het datawarehouse. De acceptatie van gebruikers is al een aantal malen benoemd als kritieke factor voor het succes van Business Intelligence.
Naast het verwachtingspatroon van de gebruikers kun je concluderen dat de kosten van het opslaan van miljarden records tegenwoordig niet meer zo groot zijn. Houd er wel rekening mee dat performance een probleem kan worden als de tabellen heel erg groot worden. Ook een slechte performance kan leiden tot slechte acceptatie van het datawarehouse. In hoofdstuk 6 zul je nog enkele trucs leren om de performance goed te krijgen.
Ook de keuze van het platform is hier belangrijk. Verschillende producten zijn geoptimaliseerd om tabellen die meerdere terabytes groot zijn goed te kunnen opslaan en met goede performance te kunnen bevragen. Dit zijn vaak wel dure producten. Voorbeelden van dit soort producten zijn Teradata, SAP HANA en Azure Synapse Analytics. Bij andere platformen loop je misschien eerder tegen beperkingen aan.
Het te kiezen detailniveau volgt als het goed is uit de informatieanalyse, uit de behoeften die bestaan binnen de organisatie. Bij twijfel moet je terug naar de gebruikers. Veel (alles) opslaan heeft over het algemeen wel de voorkeur

vanwege de sterk afgenomen kosten die dat met zich meebrengt en de daardoor toenemende mogelijkheden van het datawarehouse.

Alle beschikbare details opslaan heeft tegenwoordig de voorkeur.



Volgens Kimball is het resultaat van deze stap het zogeheten grain statement. Een grain statement zou kunnen zijn: 'Verkopen moeten worden geanalyseerd per product, per klant en per dag'.

Het **grain statement** is de uitspraak die vastlegt op welk detailniveau je gegevens in het datawarehouse gaat opslaan.



Bepaal de van toepassing zijnde dimensies

In de inleiding hebben we het begrip dimensie geïntroduceerd. Dimensies geven context, betekenis, aan de cijfers die een proces inzichtelijk maken. Een getal alleen, zonder context, betekent niets. Het is de context die de betekenis bepaalt.

De belangrijkste dimensies volgen vanzelf uit het grain statement. Bovenstaand grain statement brengt ons bij de dimensies Product, Klant en Datum. Waarschijnlijk zijn er meer dimensies dan die welke rechtstreeks uit het grain statement volgen.

De belangrijkste dimensies volgen uit het grain statement.



Het kan voorkomen dat je bovenstaand grain statement hebt geformuleerd. Uit de informatieanalyse komt ook naar voren dat mensen zitten met vragen zoals: Gaat de omzet omhoog als we een iPad weggeven bij elke aankoop van boven de € 500,-? En zo ja, geldt dat ook voor de winst, gezien de toegenomen kosten? Dergelijke vragen spelen zich niet op een lager detailniveau af dan de gedefinieerde grain. Ze leiden wel tot de introductie van een nieuwe dimensie. In figuur 4.1 is het de tabel DimPromotion die deze dimensie vertegenwoordigt.

Soms leidt de introductie van een dimensie wel tot de noodzaak een andere grain te kiezen. Deze en de voorgaande stap zijn dan ook iteratief. Je blijft ze net zo lang herhalen totdat je alle dimensies hebt gevonden.

Het bepalen van het grain statement en het bepalen van de relevante dimensies is een iteratief proces.



Dat brengt ons op de vraag wanneer iets een dimensie is.

Stel dat je uit de informatieanalyse de volgende opmerking hebt gehaald: 'Een inkoper vermoedt een verband tussen de maat van producten en de kleur'.

Kopen kinderen vaker producten in felle kleuren, terwijl volwassenen meer voor de wat neutralere kleuren gaan? Blijft dit stabiel of heeft mode en tijdgeest hier invloed op? Om beter te kunnen inkopen wil de inkoper deze vragen graag beantwoord zien.
Heb je zojuist twee nieuwe dimensies gevonden, kleur en maat? In dit geval zijn kleur en maat kenmerken van de dimensie Product. Een dimensie kenmerkt zich door de vele beschrijvende elementen die ervoor te vinden zijn. Van een product weet je zaken als inkoopprijs, verkoopprijs, levertijd, gewicht, maat en kleur. Van kleur en maat zelf weet je niets. Ze beschrijven een product maar hebben zelf geen beschrijvende kenmerken. Daarmee worden het kolommen in de tabel DimProduct in plaats van dat het zelf dimensies (en dus tabellen) worden. We noemen kleur en maat ook wel (dimensie)attributen.



Een dimensie kenmerkt zich door de vele beschrijvende elementen die ervoor te vinden zijn.

Verderop in dit hoofdstuk leer je meer over dimensies en het ontwerpen van dimensies.

Bepaal de relevante feiten

De vierde en laatste stap in het door Kimball beschreven vierstappenplan is het bepalen van de relevante feiten. Feiten worden ook wel meetwaarden (measures) genoemd. De processen, en veranderingen in die processen, moeten meetbaar zijn. Je wilt doelstellingen definiëren en kunnen meten of die doelstellingen gehaald zijn. Doelstellingen moeten SMART zijn, waarbij de M staat voor meetbaar.
Een verkoopproces is meetbaar door het aantal verkochte producten te tellen, de omzet te berekenen, de kosten en de marge te bekijken et cetera. Dat zijn de feiten, ofwel de meetwaarden. Deze feiten worden kolommen in de feitentabel, de kern van het stermodel.
In paragraaf 4.1.7 leer je meer over feiten en feitentabellen.

Beperkingen

Je hebt zojuist de vier stappen geleerd die je moet doorlopen als je een stermodel wilt ontwerpen. In theorie worden de keuzes die je tijdens dit stappenplan maakt volledig bepaald door de informatieanalyse. De gebruikers hebben kenbaar gemaakt wat ze aan informatie nodig hebben, al dan niet opgeschreven in de vorm van user stories. Dat bepaalt het nodige detailniveau, de dimensies en dimensiekenmerken, en de feiten.
Een belangrijke stap die tot nu toe onderbelicht is gebleven, is de terugkoppeling naar de gebruikte operationele systemen. Als bij een verkoop niet wordt vastgelegd of er ten tijde van die verkoop een actie gaande was, is het verhaal over van toepassing zijnde dimensies slechts theorie. Alles wat de primaire systemen niet registreren, is (waarschijnlijk) niet voorhanden.

Een dergelijke discrepantie tussen wat de gebruikers graag zouden willen en wat op basis van de bestaande gegevens mogelijk is, kan ertoe leiden dat de bestaande operationele databases als uitgangspunt gaan dienen voor het ontwerpen van je dimensionele model. Zoals uitgebreid besproken in hoofdstuk 2, is het een valkuil om Business Intelligence te zien als een technisch iets dat van ICT is. Met de bestaande databases als uitgangspunt dreigt die valkuil. Laat altijd de wensen en eisen van de gebruikers leidend zijn in je opzet. Tegelijkertijd, beloof nooit iets dat je niet kunt waarmaken.

Laat altijd de wensen en eisen van de gebruikers leidend zijn in je opzet.



4.1.3 Dimensies

Zoals gezegd geven de dimensies betekenis aan de feiten. Daarmee zijn de dimensies misschien wel de belangrijkste component van een dimensioneel model. Ze bepalen de doorsnedes die de gebruiker kan maken. Een veelgebruikte term in datawarehousing is '*to slice and dice*'. Waarbij *to slice* hier staat voor de mogelijkheid om te filteren. Beide komen neer op de wens om de cijfers vanuit verschillende invalshoeken te kunnen bekijken.

To slice and dice staat voor het maken van willekeurige doorsnedes van de feiten.



Bij het bepalen van wat dimensies zijn, kijk je in het verslag van de informatieanalyse of in user stories naar zinnen als:

- In het verkoopproces moet de omzet uitgesplitst kunnen worden naar productcategorieën.
- De gegevens moeten getoond kunnen worden voor een specifiek jaar.
- Om functioneringsgesprekken te voeren wil ik (de teamleider) per verkoper zien welke orders ze hebben gescoord en moet de resulterende omzet en marge inzichtelijk zijn.

Hoewel telkens verschillend geformuleerd, komt het iedere keer op hetzelfde neer: omzet per categorie, per jaar, per verkoper. Overal waar het woordje 'per' om de hoek komt kijken, heb je te maken met een dimensieattribuut of met een dimensie.

Een **attribuut** is een kenmerk van een dimensie, ofwel een kolom in de dimensietabel.



Uit de eerste twee genoemde zinnen volgen productcategorie en jaartal als dimensieattributen van de dimensie Product en de dimensie Datum. Je weet

niet zozeer iets over categorie of jaartal. Categorie zegt iets over een product, jaartal zegt iets over een periode.
De derde zin is iets algemener geformuleerd. Verkoper is hier een dimensie en niet een attribuut. Van een verkoper zijn veel kenmerken denkbaar, zoals naam, geboortedatum en datum in dienst. Deze kenmerken zijn de attributen van de dimensie Verkoper.
In technische termen komt het op het volgende neer. Alles waarop gefilterd of geaggregeerd moet worden, moet een attribuut zijn van een dimensie. Een attribuut is een kolom die gebruikt kan worden in de `WHERE`-clausule of in de `GROUP BY`-clausule van een query. Attributen van dimensies staan in draaitabellen op de rijen en de kolommen en worden op rapporten gebruikt in slicers en filters.



Nadat je de relevante dimensies hebt onderkend, is het zaak om zo veel mogelijk relevante beschrijvende elementen (attributen) van deze dimensies te benoemen.

Tussen de relevante dimensies vind je (in bijna alle gevallen) een datumdimensie. In bijna alle analyses die mensen doen, speelt de tijd een rol. Daarom zullen we hier nu uitgebreid aandacht besteden aan de datumdimensie.

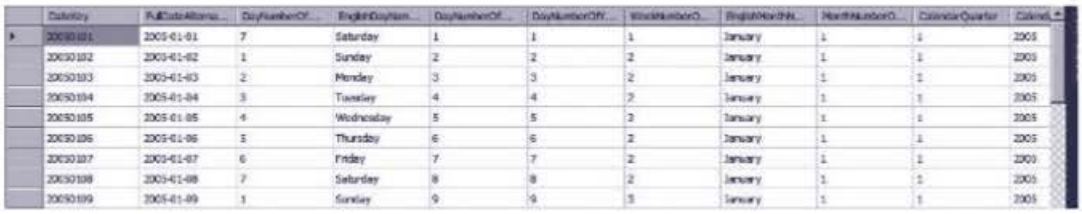
Waarom een datumdimensie?
Bijna alle datawarehouses hebben een datumdimensie. In eerste instantie is dat ook erg voor de hand liggend. In zo goed als alle gevallen speelt de tijd een rol in onze analyses. Voor een buitenstaander zegt het weinig tot niets als een bedrijf afgelopen jaar een omzet heeft gedraaid van 15 miljoen euro. De directeur heeft hier waarschijnlijk een gevoel bij: 15 miljoen is goed, of juist niet. Dat gevoel komt voort uit voorkennis, hij weet wat de omzet van het jaar ervoor was en wat de verwachting voor het afgelopen jaar was. Het gevoel komt dus uit historische kennis. De tijd speelt een belangrijke rol.



Elk datawarehouse heeft een datumdimensie, omdat de tijd een belangrijke rol speelt in ons leven.

Een belangrijk doel van Business Intelligence is om gevoel dat voortkomt uit ervaring te staven met harde feiten. Daaruit vloeit voort dat het belangrijk is om zaken als omzet te kunnen afzetten tegen periodes. Zo kunnen periodes vergeleken worden en groei en krimp berekend worden. Er is bijna geen bedrijf dat zonder dit soort vergelijkingen kan. Daarom heeft elke datamart een datumdimensie. Forecasting is ook een belangrijk proces bij veel organisaties. Dat is eigenlijk in de tijd vooruit proberen te kijken. Met trendanalyses en een beetje machine learning kunnen we met goede gegevens steeds betere voorspellingen doen. Ook hier speelt de tijd weer een belangrijke rol.
Tot nu toe spreken we hier over een datumdimensie. Dat is een tabel die voor elke dag op de kalender een rij bevat die die dag beschrijft. In figuur 4.3 zie

je een voorbeeld van een datumdimensie. Uit het grain statement moet naar voren komen of de dag genoeg is of dat het tijdstip op de dag dat iets plaatsvond ook van belang is. Je kunt een rij maken voor elke seconde van elke dag in de tabel. Vaak maak je twee aparte tabellen, een datumdimensie met een rij voor elke dag op de kalender en een aparte tijddimensie voor elke seconde binnen een dag.



Figuur 4.3 Voorbeeld van een datumdimensie

Tot hier staat beschreven wat een datumdimensie is, maar waarom je een datumdimensie nodig hebt is nog altijd niet beantwoord. Die vraag gaan we nu wel beantwoorden.

Als je in de feitentabel een kolom opneemt met daarin de datum van wanneer dat feit optrad, dus de orderdatum als het om een order-feitentabel gaat, ben je er nog niet. Met die datum kun je weliswaar berekenen wat je nodig hebt: het jaartal, het kwartaal, de maand, de naam van de dag en ga zo maar door, waarop een bepaalde order is geplaatst. Er zijn echter veel dingen niet (goed) mogelijk op basis van die datum en dat maakt dat een datumdimensie veel meerwaarde heeft in een datamart. Niet alle redenen zijn voor elke situatie van toepassing, dat moet je per project bekijken. De voordelen van een aparte datumdimensie zijn:

- gemak van filteren;
- gebroken boekjaren;
- weeknummers;
- ontbrekende data;
- extra metadata;
- performance.

We lopen de punten een voor een langs.

Gemak van filteren – In eerdere hoofdstukken is de vraag wie rapporten maakt, of beter gezegd, wie de gebruiker van het datawarehouse is, al ter sprake gekomen. Steeds vaker is de gebruiker iemand die een functie binnen het proces heeft en dus niet een ICT'er. Hoe goed is deze gebruiker in staat om uit een datum het kwartaal te halen? Jij weet misschien dat dat kan met de `DATEPART`-functie van SQL. Het onderstaande voorbeeld geeft het kwartaal terug waarin de OrderDate valt:

```
DATEPART( QUARTER, [Orderdate])
```

Niet alle gebruikers van het datawarehouse zullen over deze kennis beschikken. Ook de gebruikte tool om query's te schrijven speelt een rol. Steeds meer worden drag-and-droptools gebruikt. Als een gebruiker de omzet per kwartaal wil zien, sleept hij simpelweg de kolommen Omzet en Kwartaal naar het rapport. Klaar! Tenminste, als er een kolom Kwartaal is.



Met een datumdimensie zorg je ervoor dat er voor elke relevante periode een vooraf gedefinieerde kolom bestaat.

Gebroken boekjaren – Veel bedrijven hebben zogenaamde gebroken boekjaren. Dat wil zeggen dat een boekjaar niet loopt van 1 januari tot en met 31 december, maar bijvoorbeeld van 1 juli tot en met 30 juni. Dit heeft vaak tot gevolg dat sommige rapportages 'gewoon' de kalender volgen, maar dat andere rapportages gebaseerd moeten zijn op boekjaren. Voor een specifieke order kan dan gelden dat deze valt in kalenderjaar 2021, maar in boekjaar 2022.
Wanneer een boekjaar begint en eindigt is een bedrijfsregel (business rule). Je wilt geen bedrijfsregels opnemen in rapporten. De kans dat er fouten worden gemaakt is te groot en de controleerbaarheid en onderhoudbaarheid zijn te slecht. Voor de consistentie willen we zeker weten dat elk rapport dezelfde definitie gebruikt. Dat hoort bij level 3 en hoger in het volwassenheidsmodel van hoofdstuk 2.
Een datumdimensie met een kolom voor kalenderjaar en een aparte kolom voor fiscaal jaar of boekjaar biedt een goede structurele oplossing. Evenzo maak je kolommen voor kalenderkwartaal en fiscaalkwartaal en alle andere van toepassing zijnde periodes. De gebruiker van het datawarehouse hoeft nu alleen maar de juiste kolom te gebruiken in zijn analyse.



De datumdimensie bevat de bedrijfsregel die definieert van wanneer tot wanneer een (gebroken) boekjaar loopt.

Weeknummers – Een ander veel terugkerend probleem is het weeknummer. Is week 1 nou de week waarin 1 januari valt, de week waarin ten minste vier dagen van januari voorkomen of de eerste volle week in januari? Er zijn veel definities mogelijk en veel mensen weten eigenlijk niet eens welke definitie de standaard is in Nederland. In de datumdimensie kun je voor elke dag opnemen in welke week die dag valt.



De datumdimensie bevat de bedrijfsregel die definieert in welk weeknummer een dag valt.

Los van de business rules ten aanzien van boekjaren en weeknummers zijn er nog andere tijdgerelateerde business rules denkbaar. Ook deze kunnen in de datumdimensie worden opgenomen.

Ontbrekende data – Stel, je maakt een datamart voor een kleine ICT-opleider. Omdat er in de zomer bijna niemand op cursus komt, is de opleider de hele maand augustus gesloten. Stel verder dat je geen datumdimensie hebt maar alleen een kolom met de startdatum van elke cursus. Als iemand met deze gegevens een staafgrafiek maakt om de omzet per maand te tonen, ontbreekt de hele maand augustus. De grafiek heeft maar elf staven terwijl een jaar twaalf maanden kent.
Ook hier geldt weer dat een datumdimensie de oplossing kan bieden. Voor elke periode die op de kalender voorkomt maak je een record, ongeacht of je in die periode zaken hebt gedaan. Feiten of niet, een dag is een dag. En het feit dat je in een bepaalde periode geen business hebt gehad, is informatie op zich.



Een datumdimensie bevat gegevens over alle periodes, ook periodes waarin zich geen feiten hebben voorgedaan.

Extra metadata – Los van hele dagen die in operationele databases niet voorkomen, is er vaak extra kennis over een dag nodig. Is een dag een werkdag? Zo niet, is het een weekenddag, of is het een vakantiedag zoals Koningsdag? Je kunt een datumdimensie verrijken met dit soort additionele informatie. De gebruiker kan nu het effect van vakantiedagen op de omzet analyseren.



Een datumdimensie bevat extra informatie over periodes, bijvoorbeeld of een dag een werkdag is.

Performance – Tot slot is er nog een performancereden om een datumdimensie te maken. Deels heeft dat te maken met het eerste argument. In plaats van voor elke query opnieuw met functies zoals de getoonde `DATEPART` een periode te berekenen, is die berekening eenmalig, vooraf, gedaan. Dat spaart rekentijd. Bovendien is er een grote kans dat de database-engine, het proces dat de query moet uitvoeren, efficiënter kan werken met de kolommen uit de datumdimensie. Je query's worden dus sneller.

Hoe bouw je een datumdimensie?

Nu je weet waarom je een datumdimensie nodig hebt in een datamart, is het tijd om er één te maken. Dat doe je in drie stappen:

1. Bepaal het gewenste detailniveau.
2. Bepaal de sleutels.
3. Bepaal de overige kolommen.

Bepaal het gewenste detailniveau – De eerste stap is het bepalen van het gewenste detailniveau. Als het goed is volgt dat uit het grain statement van het stermodel. In de praktijk zijn veel datumdimensies op dagniveau. Dat houdt in dat je tot op dagniveau analyses kunt doen. Je maakt hiertoe voor elke dag op de kalender een record in de datumdimensie.
In sommige datamarts moeten de feiten tot op de seconde terug geanalyseerd kunnen worden. Dit houdt volgens hetzelfde principe in dat je voor elke mogelijke seconde een record aanmaakt. Als je tien jaar aan historie wilt gaan opslaan, krijg je ruim 315 miljoen records. Waarschijnlijk is het aan te raden om een aparte datum- en tijddimensie aan te maken. Deze bevatten respectievelijk ruim 3600 en ruim 86.000 records.



In veel datawarehouses bevat de datumdimensie een record voor elke dag op de kalender.

Bepaal de sleutels – De volgende stap is het bepalen van de sleutels. In eerste instantie moet je een primaire sleutel (primary key) kiezen. Het is slim om daarnaast ook sleutels te maken voor periodes.
Voor de datumdimensie is het een best practice om een zogenaamde smart key te gebruiken. Een smart key is niet een betekenisloos nummertje maar bevat echte informatie. De sleutel van een datumdimensie ziet er bij voorkeur uit zoals in onderstaand voorbeeld:

```
20210611
```

Dit is een geheel getal en wordt ook als zodanig opgeslagen in de datumdimensie. Het vergt niet veel fantasie om in dit getal de datum 11 juni 2021 te herkennen. Het voordeel van een sleutel als deze is dat gehele getallen efficiënter zijn in opslag en een betere performance opleveren dan datums. Door eerst het jaartal op te nemen en dan pas maand en dag, blijft wel de juiste sorteervolgorde gehandhaafd.
Het bovenstaande is niet voor alle databaseproducten waar. In sommige producten is het gebruik van een echte datum in plaats van de hier voorgestelde smart key beter. Houd met het opzetten van een database altijd rekening met het specifieke product dat je gaat gebruiken om de database te implementeren.
Naast de primaire sleutel, de kolom die voor elke rij een unieke waarde bevat, zijn er andere sleutels noodzakelijk. Stel dat je in je datumdimensie een kolom Kwartaal hebt opgenomen en dat daarin de getallen 1 tot en met 4 kunnen voorkomen. In figuur 4.4 kun je zien wat er gebeurt als iemand een draaitabel maakt met de kwartalen op de horizontale as.

	A	B	C	D	E	F
1	Internet Sales Amount	Column Labels				
2	Row Labels	1	2	3	4	Grand Total
3	⊞Accessories	$173,550.63	$199,754.98	$152,419.17	$175,035.18	$700,759.96
4	⊞Bikes	$7,229,254.22	$8,776,282.57	$5,420,034.74	$6,892,573.11	$28,318,144.65
5	⊞Clothing	$86,053.86	$98,374.79	$73,083.70	$82,260.26	$339,772.61
6	Grand Total	$7,488,858.71	$9,074,412.34	$5,645,537.61	$7,149,868.55	$29,358,677.22

Figuur 4.4 Draaitabel

Los van het feit dat een kolom met daarin het getal 1 niet zoveel zegt – Q1 of kwartaal 1 was beter geweest – zijn alle feiten waarvoor geldt dat de kolom Kwartaal een 1 bevat, bij elkaar opgeteld. Ongeacht het jaartal. De getoonde draaitabel vergelijkt de omzet per kwartaal over alle jaren heen. Misschien dat dit de bedoeling is, maar de kans is groot dat de gebruiker twaalf kolommen wil zien als hij drie jaargangen selecteert, voor elk jaar vier kwartalen.
In feite toont figuur 4.4 twee problemen:

- niet goed leesbare kolomheaders;
- (mogelijk) een inconsistente analyse.

Een goede datumdimensie kan beide problemen oplossen. De datumdimensie moet een kolom KwartaalKey bevatten volgens hetzelfde principe als de smart key die je hebt gemaakt als primary key. Je krijgt dan een kolom met als inhoud bijvoorbeeld `20212`, voor het tweede kwartaal van het jaar 2021. Deze sleutel zorgt dat elk kwartaal een unieke sleutel heeft en dus dat elk afzonderlijk kwartaal getoond wordt op rapportages. De sleutel wordt weer als geheel getal opgeslagen in de dimensietabel.
Naast de sleutel kun je ook nog een beschrijvende kolom maken. Hierin kun je bijvoorbeeld de tekst '`Q2, 2021`' zetten. Als de gebruiker deze kolom sleept naar een rapport, zullen de headers meteen leesbaar en intuïtief zijn. En dat is goed voor de acceptatie van het datawarehouse.

> Een goede datumdimensie bevat voor elke relevante periode een unieke sleutel in de vorm van een getal en een kolom met een leesbare beschrijving.



Bepaal de overige kolommen – Naast de sleutels bevat de datumdimensie beschrijvende informatie. Je moet hierbij denken aan de naam van de dag, het weeknummer waarin die dag valt, de maand van die dag, zowel als nummer als met naam (dus maandnummer is 1 en maandnaam is januari), of de dag een werkdag is, enzovoort.

Nu je een idee hebt welke kolommen je moet maken (allemaal uiteraard gebaseerd op de informatieanalyse) kun je de tabel daadwerkelijk maken. Als je op Google zoekt naar 'create date dimension with Excel' vind je veel hits. Ook Kimball heeft een generator voor een datumdimensie, gebouwd in Excel, die je kunt downloaden van zijn website. In de Office 2013 appstore van Microsoft

vind je een vergelijkbare generator. Googlen op SQL en date dimension levert ook verschillende hits op voor SQL-scripts die op verschillende platformen datumdimensies genereren. Op de website bij dit boek vind je een T-SQL-script om een datumdimensie te bouwen in SQL Server. Er zijn nog vele andere manieren om een datumdimensie te genereren. Afhankelijk van waar je een datumdimensie voor nodig hebt, kies je de makkelijkste manier. Voor een relationeel datawarehouse is het uitvoeren van een SQL-script waarschijnlijk het meest voor de hand liggend.



Scripts om datumdimensies te genereren zijn op veel plekken te downloaden.

Overige dimensies

Zoals gezegd is de kans zeer groot dat een datamart een datumdimensie zal bevatten. Over de andere dimensies is op voorhand uiteraard veel minder te zeggen. In de gezondheidszorg heb je misschien een patiënt- of cliëntdimensie, bij een commerciële webshop eerder een klantdimensie. Er is echter wel een aantal algemene zaken te noemen met betrekking tot het maken van dimensies. We zullen hier noemen:

- het aantal te maken dimensies;
- denormaliseren van dimensies;
- de te kiezen sleutels;
- het soort informatie.

Het aantal te maken dimensies

Als nadeel van normaliseren is eerder in dit boek genoemd dat er erg veel tabellen ontstaan. Hoeveel dimensies (en dus hoeveel tabellen) heeft een 'goed' stermodel?

Dit antwoord is allereerst volledig afhankelijk van de wensen en eisen. De gevraagde functionaliteit moet geboden kunnen worden. Los daarvan bestaat er een soort vuistregel dat er niet meer dan zeven dimensies per ster moeten bestaan. Het getal zeven komt uit de psychologie. Uit onderzoek blijkt dat mensen zeven zaken tegelijkertijd in hun hoofd kunnen hebben zonder het overzicht te verliezen. Laat iemand zeven plaatjes zien en vraag later terug welke plaatjes het waren, en de persoon is goed in staat om de getoonde plaatjes op te noemen. Maak er acht plaatjes van en de kans wordt erg groot dat iemand er één vergeet. Intuïtiviteit van de datamart is belangrijk voor de acceptatie. Gebruikers moeten het overzicht kunnen behouden. Vandaar de vuistregel van zeven dimensies.



Probeer het aantal dimensies met stermodel te beperken tot maximaal zeven.

Zeven klinkt erg weinig, zeker als je genormaliseerde databases gewend bent met honderden of zelfs duizenden tabellen. Toch blijkt dat in de praktijk erg mee te vallen. Voor elk proces zijn de eerste vier tot vijf dimensies zo bedacht. Daarna blijkt het al snel lastig te worden om nog meer zinvolle dimensies te verzinnen. Vaak zal wat je verzint een attribuut van een al bestaande dimensie zijn.

Denormaliseren van dimensies

Misschien is het je opgevallen dat in de datumdimensie zoals eerder besproken, veel redundantie voorkomt. In de dimensie zoals getoond in figuur 4.3 zijn eigenlijk alle kolommen redundant behalve de eerste of de tweede. Alle kolommen kunnen berekend worden uit de kolom Datum en daarmee zijn alle kolommen redundant. Voor dimensietabellen is redundantie normaal. Dimensies zijn platgeslagen, of in andere woorden, gedenormaliseerd.

Dimensietabellen zijn platgeslagen, niet-genormaliseerde tabellen die veel redundantie bevatten.

Voorbeeld 4.2

Stel bijvoorbeeld weer dat je de datamart bouwt voor een kleine ICT-opleider. Deze opleider verzorgt cursussen voor zijn klanten. Klanten zijn bedrijven, de cursisten zijn individuen en werknemers bij de klant. In een genormaliseerde database zou je twee tabellen maken, klanten en cursisten. De tabel klanten bevat gegevens over de bedrijven die cursisten sturen en die de facturen betalen. De tabel cursisten bevat NAW-gegevens van de individuele cursisten. In een dimensie voeg je dit samen tot één tabel. Dat wordt een platgeslagen tabel genoemd. Zie figuur 4.5.

Cursist-Key	Naam	Geboortedatum	Email	Telefoon	Bedrijf	KVK-nummer	BTW-nummer	Factuuradres
1	Jan	1-1-1980	Jan@trainsql.nl	06-12345	TrainSQL	123	55555	Dorpsstraat
2	Piet	2-2-1988	Piet@trainsql.nl	06-54321	TrainSQL	123	55555	Dorpsstraat
3	Klaas	3-3-1985	Klaas@topbi.nl	06-98765	TopBI	456	66666	Kerkstraat
...	...	...	...	...	...			...

Figuur 4.5 Cursistdimensie

Wat opvalt in de tabel van figuur 4.5 is dat de informatie met betrekking tot het bedrijf TrainSQL tweemaal is opgeslagen omdat er twee cursisten, Jan en Piet, door TrainSQL op cursus zijn gestuurd. Als er drie cursisten waren gestuurd, waren de gegevens driemaal opgeslagen. Dit is redundant en lijkt daarmee overbodig.

Bedenk echter hoe de tabel gebruikt wordt. Een gebruiker kan de kolom Bedrijf slepen naar een as in een draaitabel of naar een filter in zijn rapport. Automatisch wordt alles van hetzelfde bedrijf samengevoegd. Dus zaken als omzet per klant zijn gemakkelijk (en met drag-and-drop) op te vragen. Er hoeven geen (moeilijke) joins geschreven te worden en de database-engine verspilt geen moeite aan het oplossen van die joins.

De te kiezen sleutels

Het is best practice om dummy keys te introduceren in je datamart. Deze dummy keys worden surrogate keys of datawarehouse keys genoemd. Het zijn willekeurige nummers, uitgedeeld tijdens het schrijven van nieuwe records in de dimensie, en ze dienen om records uniek te maken.



Een **surrogate key** of **datawarehouse key** is een automatisch gegenereerd, uniek, betekenisloos nummer dat elke record krijgt toegewezen op het moment dat het in het datawarehouse wordt weggeschreven.

Meestal is het van belang om van elk record in de datamart te kunnen herleiden welk record in de operationele database eraan ten grondslag ligt. Daarom bevat een dimensie ook de primaire sleutel van het bronrecord. Dit veld wordt vaak alternate key, source key of, in het Nederlands, bronsleutel, genoemd. In de tabel zoals beschreven in figuur 4.5 zou dit tot twee extra velden leiden: CursistSourceKey en KlantSourceKey.

Het soort informatie

Zoals al een aantal keren gezegd, geven de dimensies context aan de feiten. Ze geven de feiten betekenis. Hoe meer beschrijvende velden, hoe groter de meerwaarde van de dimensie. Hoe verhoudt het aantal cursisten met een mbo-achtergrond zich tot het aantal cursisten met een hbo- of wo-achtergrond? Met een attribuut (kolom) vooropleiding in de cursistdimensie, kun je deze vraag beantwoorden. Hoe ver zijn cursisten bereid te reizen voor een cursus en is dat gelijk voor alle cursussen of verschilt dat per cursus? Met een attribuut woonplaats (en een cursusdimensie) is ook deze vraag te beantwoorden.
Hoe meer attributen er zijn, hoe meer analyses je kunt doen. De attributen moeten bij voorkeur tekstuele beschrijvingen zijn en geen codes, tenzij die codes zijn doorgedrongen tot het dagelijks taalgebruik van het bedrijf. Denk maar eens terug aan het verhaal over kwartalen in de datumdimensie. Codes maken slechte kolomheaders op rapporten. En met draaitabellen vormt de inhoud van je kolommen de headers van je draaitabel.



Een dimensie bevat bij voorkeur zoveel mogelijk tekstuele informatie met betrekking tot de dimensie.

Samenvatting

- Dimensies spelen een belangrijke rol bij dimensioneel modelleren. Bij het ontwerpen van de dimensietabellen moet je een paar richtlijnen goed voor ogen houden.
- Gebruik surrogate keys voor alle dimensies, behalve voor de datumdimensie. Gebruik een smart key voor de datumdimensie in de vorm van yyyymmdd. Dus bijvoorbeeld 20210630.
- Neem naast de surrogate key ook de sleutel van het bijbehorende record uit het bronsysteem op.
- Voeg zo veel mogelijk beschrijvende kolommen toe aan de dimensies.
- Ga niet normaliseren maar maak platgeslagen tabellen.
- Beperk het aantal dimensietabellen in een stermodel tot ongeveer zeven.

Nu je weet wat dimensietabellen en dimesieattributen zijn, is het tijd om naar geavanceerdere zaken met betrekking tot dimensies te kijken. We beginnen met het bespreken van historie. Hoe houdt een datawarehouse historie van gegevens bij?

4.1.4 Slowly Changing Dimensions (SCD)

In hoofdstuk 3 is als een van de mogelijke problemen met rapporten op operationele databases het gebrek aan historische gegevens genoemd. Kimball beschrijft dit als Slowly Changing Dimensions (SCD) en komt met enkele pasklare oplossingen.
De term 'Slowly' refereert hier vooral aan het feit dat sommige gegevens slechts af en toe veranderen. Het gaat om ad-hoc-wijzigingen. Denk bijvoorbeeld aan patiënten die verhuizen. Dat gebeurt af en toe, maar zeker niet dagelijks, laat staan meerdere keren per dag. Soms kan de frequentie van dit soort wijzigingen wel groot zijn. Dat heeft impact op de oplossing die je moet kiezen.
Met de term 'Changing Dimension' moet je een beetje oppassen. Eigenlijk wordt bedoeld een wijziging in een dimensieattribuut. Een dimensie bestaat uit vele attributen. Per attribuut moet je bedenken wat je wilt doen als de waarde (inhoud) van zo'n attribuut verandert. De keuze komt uiteraard voort uit de informatieanalyse omdat de keuze bepaalt welke functionaliteit je krijgt.



De term **Slowly Changing Dimension** refereert aan het feit dat attributen van dimensies in de loop van de tijd kunnen veranderen en draagt standaardoplossingen aan voor hoe je hier in het datawarehouse mee om kunt gaan.

Kimball beschrijft in zijn oorspronkelijke theorie drie manieren om met veranderingen van dimensieattributen om te gaan. Dit zijn:

- SCD type 1
- SCD type 2
- SCD type 3

Als je zoekt op Wikipedia, vind je daar ondertussen acht SCD-types, plus de opmerking dat je de verschillende oplossingen kunt combineren. We zullen hier alleen de drie standaardoplossingen bespreken.

Slowly Changing Dimension type 1

Denk nog eens terug aan de cursistdimensie uit de vorige paragraaf. Een van de kolommen is het e-mailadres van de cursist. Hoewel het een zinvol attribuut is, zal het e-mailadres niet gebruikt worden om in een draaitabel een as te vullen of om gegevens te filteren. Omzet per e-mailadres is geen relevante analyse.
In het bovenstaande voorbeeld is het vooral nuttig om het actuele e-mailadres van cursisten bij de hand te hebben. Als een cursist een nieuw e-mailadres krijgt, overschrijf je het oude adres gewoon met het nieuwe. Dit is hoeveel operationele systemen ook omgaan met wijzigingen. Binnen het dimensioneel modelleren wordt dit een type 1 Slowly Changing Dimension genoemd.



Bij **SCD type 1** wordt de oude waarde van een attribuut overschreven door de nieuwe, actuele, waarde. De waarde van het attribuut wordt niet historisch bijgehouden.

Je kunt er natuurlijk ook voor kiezen om een wijziging niet door te voeren. Als je het oude e-mailadres behoudt, en negeert dat er een meer actuele waarde voor e-mailadres bestaat, heet dat een type 0 SCD.

Slowly Changing Dimension type 2

Het attribuut Bedrijf in de cursistdimensie levert een ander verhaal op. Als je bijvoorbeeld de omzet per klant (kolom Bedrijf in het voorbeeld) berekent, en je hebt net zoals bij het e-mailadres de oude waarde overschreven door de nieuwe waarde, dan staan alle cursussen die deze persoon heeft gevolgd, vanaf nu geregistreerd bij zijn nieuwe werkgever. Dit is het probleem zoals eerder beschreven in paragraaf 3.7.
Voor dit soort situaties is Slowly Changing Dimension type 2 bedoeld. Bij een SCD type 2 maak je bij elke verandering een compleet nieuw record aan. Nadat cursist Jan uit figuur 4.5 van werkgever is veranderd, ziet de cursistdimensie eruit zoals in figuur 4.6. Alle kolommen die iets zeggen over Jan, zoals zijn geboortedatum, veranderen niet. Die worden onveranderd overgenomen. Alleen het bedrijf verandert (en in dit voorbeeld alle kolommen die iets zeggen over de klant).

Cursist-Key	Naam	Geboortedatum	Email	Telefoon	Bedrijf	KVK-nummer	BTW-nummer	Factuuradres
1	Jan	1-1-1980	Jan@trainsql.nl	06-12345	TrainSQL	123	55555	Dorpsstraat
2	Piet	2-2-1988	Piet@trainsql.nl	06-54321	TrainSQL	123	55555	Dorpsstraat
3	Klaas	3-3-1985	Klaas@topbi.nl	06-98765	TopBI	456	66666	Kerkstraat
...	...		...	...	...			...
1001	Jan	1-1-1980	Jan@bedrijfA.nl	06-34567	BedrijfA	789	77777	Ergens

Figuur 4.6 Cursistdimensie met SCD 2

Alle keren dat Jan in het verleden op cursus is geweest, staan vermeld in de feitentabel met CursistKey 1. Die feiten zijn dus gelinkt aan het eerste record en daarmee aan de persoon Jan, maar tegelijkertijd ook aan het bedrijf TrainSQL. Vanaf nu zal CursistKey 1001 gebruikt worden als Jan op cursus komt. Nieuwe rijen in de feitentabel worden dus gekoppeld aan rij 1001. Dat is nog steeds persoon Jan maar nu bedrijf BedrijfA. Als je een analyse doet op omzet per klant (Bedrijf), blijft de oude omzet bij het oude bedrijf omdat die omzet hoort bij CursistKey 1. De nieuwe feiten worden gekoppeld aan CursistKey 1001 en vallen dus onder BedrijfA.

Bij **SCD type 2** wordt bij elke verandering van een attribuut een volledig nieuw record aangemaakt. Er bestaan 'actuele' records en 'afgesloten' records.



Merk op dat in de vorige paragraaf is gezegd dat het verstandig is om ook de SourceKey (unieke sleutel in het bronsysteem) op te nemen. Door te analyseren op deze SourceKey kun je nog steeds zaken als omzet per cursist opvragen. Zowel omzet per klant als omzet per cursist kunnen nu correct opgevraagd worden, ondanks dat er cursisten zijn die in de loop van de tijd voor andere klanten gaan werken.
Over het algemeen worden bij een SCD type 2 nog wat extra kolommen aan de dimensie toegevoegd. Dat zijn kolommen met een ingangsdatum en een einddatum en wellicht een kolom IsHuidig, die aangeeft of dit record het meest actuele record betreft. Deze kolommen helpen bij het ETL-proces en bij vragen als: hoeveel klanten en hoeveel individuele cursisten hebben wij? Het simpel tellen van records is nu niet meer genoeg!

Slowly Changing Dimension type 3

Stel, je categoriseert klanten in de categorieën zilver, goud en platina op basis van de gegenereerde omzet over de afgelopen twaalf maanden. Elke maand kunnen klanten dus in een andere categorie terechtkomen. Op basis van de

informatieanalyse is gebleken dat er behoefte is aan inzicht in hoeveel klanten van zilver worden tot goud of platina en vice versa. Stel je bij dit voorbeeld voor dat je in de eerder gebruikte tabel een kolom Categorie hebt toegevoegd.
Voor de kolom Categorie moet je historie bijhouden, anders kun je nooit achterhalen welke, laat staan hoeveel, klanten er per maand van categorie veranderen. Met een SCD type 2 is dit soort vragen wel op te lossen. Alleen levert dat heel complexe, en daarmee trage, queries op. Je moet immers in een en dezelfde tabel voor elk record op zoek naar het meest recente vorige record met dezelfde SourceKey waarvoor geldt dat de kolom Categorie is veranderd. Het meest recente afgesloten record ophalen is al een karwei. Maar dat record kan zijn voor een andere SCD type 2-kolom. Het moet wel gaan om een wijziging van de categorie. Probeer maar eens zo'n query te schrijven.
Voor dit soort vraagstukken is Slowly Changing Dimension type 3 bedacht. Bij type 3 maak je twee kolommen aan voor categorie, de kolom HuidigeCategorie en de kolom VorigeCategorie, alsmede een kolom met daarin de datum van de laatste wijziging. De benodigde informatie zit nu in zijn geheel in het record zelf. Het schrijven van de query is nu voornamelijk het definiëren van de juiste filters. Daarmee is de analyse eenvoudig geworden en tegelijkertijd snel.



Bij **SCD type 3** houd je van een attribuut in twee verschillende kolommen zowel de huidige als de vorige waarde bij. Op die manier kunnen huidige en vorige waarde gemakkelijk vergeleken worden.

Uiteraard zit er wel een beperking aan deze oplossing. Je weet de huidige en de vorige categorie, maar in welke categorie zat de klant daarvoor? Je kunt een derde, een vierde, een vijfde kolom toevoegen, maar dat houdt ergens op. De informatieanalyse moet duidelijk maken of type 3 een goede oplossing is voor de gevraagde informatie of dat deze oplossing te beperkt is.

Samenvatting

Slowly Changing Dimensions bepalen hoe je omgaat met veranderende gegevens. Per dimensieattribuut moet je bepalen of het historisch bijhouden van gegevens nodig is en welke manier het beste aansluit bij de gevraagde functionaliteit. Door het bijhouden van historie in een dimensioneel model kun je meer soorten analyses uitvoeren met de beschikbare gegevens dan mogelijk is met de gegevens uit een operationele database die zelf geen historie bijhoudt van de gegevens. Bij het laden van de gegevens in het datawarehouse moet gecontroleerd worden of er gegevens zijn veranderd en daar moet op de juiste manier op gereageerd worden.

Nu je weet hoe je in een dimensioneel model historie bijhoudt, is het tijd om te leren wat conformed dimensions zijn en waar ze voor dienen.

4.1.5 Conformed dimensions en snowflakes

Als je nog eens terugkijkt naar figuur 4.1 in de inleiding van dit hoofdstuk, zie je naast de tabel DimProduct ook de tabellen DimProductSubcategory en DimProductCategory. Er is hier sprake van een zogenaamde snowflake in plaats van een puur stermodel. Kimball spreekt van een snowflake als een of meer dimensietabellen zijn uitgenormaliseerd. In paragraaf 4.1.3 staat dat dimensies zijn platgeslagen en (daardoor) veel redundantie bevatten. Dat is normaal gesproken goed voor de eenvoud en de performance van de datamart.



Een **snowflake** is een stermodel waarvan een of meer dimensies niet zijn platgeslagen maar zijn genormaliseerd.

In het voorbeeld van figuur 4.1 is er sprake van een-op-veelrelaties tussen categorieën, subcategorieën en producten. Een categorie bestaat uit meer subcategorieën, maar een subcategorie is onderdeel van slechts een categorie. Tussen producten en subcategorieën bestaat zo'n zelfde relatie. Volgens de regels van het normaliseren zou je drie tabellen moeten hebben zoals je ook ziet in het ERD van figuur 4.1. Volgens de theorie van Kimball zou je van deze drie tabellen één tabel moeten maken. Bij elke rij in de tabel DimProduct kun je alle relevante informatie over de categorie en de subcategorie waarin een product valt, opnemen. De drie tabellen samenvoegen tot een noemen we het platslaan van de tabellen. Bij een snowflake-model volg je niet de pure theorie van dimensioneel modelleren, maar worden een of meer dimensies toch genormaliseerd. Twee belangrijke redenen om dat te doen zijn:

- het maken van conformed dimensions;
- beheersbaarheid.

Maken van conformed dimensions

Tot nu toe richt het verhaal zich op het ontwerpen van één ster. Het totale datawarehouse bestaat in de theorie van Kimball uit alle sterren die alle processen beschrijven. Veel dimensies zullen voor meer dan één proces interessant zijn. Je wilt deze dimensies zoveel als mogelijk hergebruiken bij alle relevante processen.

Een eerste stap in het hergebruik van dimensies is het opstellen van een dimensiematrix. In een dimensiematrix zet je de processen uit tegen de dimensies om gemakkelijk te kunnen zien welke dimensies relevant zijn voor welke processen. Figuur 4.7 is een voorbeeld van een dimensiematrix.

Proces	Dimensie			
	Datum	Cursus	Cursist	Locatie
Verkoop	X	X	X	X
Targeting	X	?		X
Evalueren	X	X	X	X

Figuur 4.7 Een dimensiematrix



Een **dimensiematrix** laat overzichtelijk zien welke dimensies relevant zijn voor welke processen.

De kruisjes in de matrix laten zien welke dimensie relevant is voor welk proces. Bij Targeting staat echter een vraagteken. Stel dat alle cursussen zijn ingedeeld in curricula, een Business Intelligence-curriculum, een databasebeheer-curriculum, een programmeer-curriculum enzovoort. Stel verder dat bij het bepalen van de doelstellingen voor een jaar niet een doelstelling wordt geformuleerd voor elke individuele cursus, maar een doelstelling per curriculum. Curricula worden echter niet verkocht, het zijn de individuele cursussen die worden verkocht.

Voor beide processen, verkoop en targeting, is de cursusdimensie van belang. De grain van de processen is echter anders. Voor verkoop is de grain de cursus. Per cursus moet omzet bepaald kunnen worden. Voor targeting is curriculum de grain. Per curriculum en niet per individuele cursus wordt een target bepaald. Deze situatie is een goede reden om de gegevens omtrent het curriculum niet op te nemen in de dimensie Cursus, maar de curriculuminformatie af te splitsen, te normaliseren. Je krijgt nu een dimensie Cursus en een aparte dimensie Curriculum met een één-op-veel relatie tussen beide.

Voor de ster die behoort bij het verkoopproces heb je nu een paar opties. Je kunt de CurriculumKey opnemen in de cursusdimensie. Je krijgt nu een snowflake. Dit is wat er in figuur 4.1 is gebeurd met categorie en subcategorie. Je kunt ook bij elke verkoop zowel een CursusKey als een CurriculumKey opnemen. Er is nu nog steeds sprake van een stermodel. Combinaties zijn natuurlijk ook mogelijk.

Kimball spreekt in bovenstaand geval over conformed dimensions. Daar wordt niets anders mee bedoeld dan dat een dimensie zo is ontworpen dat hij bruikbaar is voor elk proces waarvoor hij relevant is. Je moet dus proberen om zo generiek mogelijke dimensies te maken.



Een **conformed dimension** is een dimensie die dusdanig generiek is opgezet dat hij door elk relevant stermodel, zonder aanpassingen, gebruikt kan worden.

Beheersbaarheid

Beheersbaarheid is het tweede argument om dimensietabellen te normaliseren. Denk nog eens terug aan het verhaal over Slowly Changing Dimensions. In de cursistdimensie staat informatie over de individuele cursist, maar ook over het bedrijf (de klant) waarvoor deze persoon werkt. Stel dat tussen deze gegevens SCD type 2-kolommen zitten. Een klant kan misschien wel honderden cursisten gestuurd hebben (denk bijvoorbeeld aan een grote bank waar iedereen op cursus gaat om de nieuwe mogelijkheden van een nieuwe Office-versie te leren kennen). Eén wijziging in een SCD type 2-kolom zou leiden tot honderden nieuwe records. Een aparte dimensie Klant, naast de dimensie Cursist, lijkt nu toch een goed idee.



Het normaliseren van een dimensie en daarmee het maken van een snowflake kan helpen om dimensietabellen klein te houden.

4.1.6 Overige dimensie-overwegingen

Er valt nog veel meer te zeggen over goed dimensieontwerp. De volledige theorie van dimensioneel modelleren vind je in *The Data Warehouse Toolkit* van Kimball en Ross. Hier zullen we onderstaande veelgebruikte constructies en termen nog noemen:

- junkdimensie;
- gedegenereerde dimensie;
- unknown member.

Junkdimensies

Stel je weer het opleidingsinstituut van eerdere voorbeelden voor. Dit opleidingsinstituut hanteert een strippenkaart. Vooraf kan een bedrijf tegen leuke korting honderd cursusdagen inkopen. Elke keer dat een cursist een dag op cursus komt, gaat er een strip van het tegoed af. Van elke inschrijving wordt bijgehouden of het betaald is via een strippenkaart of juist niet. Verder heeft elke inschrijving een status: definitief of geannuleerd.

Je kunt in dit geval een dimensie Status maken en een dimensie TypeBetaling. Elke dimensie bevat slechts twee records. Het model wordt met twee aparte dimensietabellen complexer dan nodig. De zogenaamde junkdimensie kan uitkomst bieden. Je voegt beide kenmerken samen in een tabel met alle mogelijke combinaties. Vanuit elk feitenrecord verwijs je naar de juiste combinatie. Figuur 4.8 is een voorbeeld van een junkdimensie.

JunkKey	Strippenkaart	Status
1	Ja	Definitief
2	Ja	Geannuleerd
3	Nee	Definitief
4	Nee	Geannuleerd

Figuur 4.8 Junkdimensie



Een **junkdimensie** is een dimensie die bestaat uit een combinatie van enkele niet-gerelateerde, zeer kleine, dimensies.

Door kleine dimensies te combineren beperk je het aantal dimensietabellen waaruit het stermodel bestaat. Daarmee houd je het model overzichtelijker en is het makkelijker om je aan de best-practice te houden van maximaal zeven dimensies. Intuïtiviteit van het model is een belangrijk kenmerk van een goed stermodel.

Degenerate dimensie

Soms zijn er dimensies die geen attributen hebben. Denk bijvoorbeeld eens aan orders. Het ordernummer is een dimensieattribuut. Als je tot op individuele orders analyses wilt doen en de bijbehorende ordernummers wilt tonen, moeten deze in een dimensie zitten. Dus maak je een dimensie Order. Welke attributen zitten daarin behalve het ordernummer?
Alle relevante zaken met betrekking tot de order zijn waarschijnlijk al onderdeel van andere dimensies. Zaken als orderdatum zitten in de dimensie Datum. Alle klantgegevens zitten in de dimensie Klant.
Een degenerate dimensie is een dimensie zonder kolommen zoals de hiervoor genoemde dimensie Order. Vaak wordt hier niet een aparte dimensietabel voor gemaakt. Je kunt het ordernummer opnemen als kolom in de feitentabel.



Een **gedegenereerde dimensie** is een dimensie zonder dimensieattributen.

Unknown en niet van toepassing

In hoofdstuk 3 is als een van de argumenten ter verantwoording van een datawarehouse de soms slechte kwaliteit van gegevens in operationele databases genoemd. Het blijkt in het cursistregistratiesysteem van de opleider uit het voorbeeld voor te komen dat er inschrijvingen bestaan van niet (meer) bekende personen. Als je query's schrijft op een database waarin dat soort zaken voorkomt, moet je je daar goed bewust van zijn. Waarschijnlijk moet je een `OUTER JOIN` schrijven om de juiste resultaten te krijgen.
Over het algemeen is dat voor een datamart zeer ongewenst. Drag-and-drop tools genereren waarschijnlijk de verkeerde query's. Om problemen te voorkomen is het normaal om in elke dimensie een extra record op te nemen, de

Unknown. Deze krijgt als Key meestal de waarde –1. Het is eigenlijk een dummy record. Hij zorgt ervoor dat ook feiten met ontbrekende dimensiegegevens meegenomen worden in rapporten.



Unknown is een dummy record in een dimensietabel. Dit record zorgt ervoor dat feiten uit de feitentabel die niet gelinkt kunnen worden aan een dimensierecord nu toch gelinkt kunnen worden en daarmee correct getoond worden op rapporten.

Naast dat het juiste dimensierecord onbekend kan zijn, zou het ook kunnen zijn dat een dimensie niet van toepassing is. Stel, je hebt een feitentabel met verkopen. Je houdt door middel van een dimensie Verkoper bij wie welke verkopen heeft gerealiseerd. Voor verkopen via het internet is er echter helemaal geen verkoper. Internetverkopen laten verwijzen naar Unknown is raar. Het is helemaal niet onbekend, het is simpelweg niet van toepassing.
In dit soort situaties wordt vaak een tweede dummy record opgenomen in de dimensietabel. Dit is het record 'Niet van toepassing' en deze krijgt over het algemeen –2 als waarde voor de Key.

Je weet al veel over dimensies. Daarom is het nu tijd om meer te weten te komen over feiten en de feitentabel.

4.1.7 Feiten

Bij paragraaf 4.1.3 over dimensies heb ik gezegd dat dimensies misschien wel de belangrijkste component van je dimensioneel model zijn vanwege de context die ze geven aan de feiten. Maar uiteindelijk willen mensen feiten zien. De processen moeten inzichtelijk gemaakt worden. Inzichtelijk houdt in meetbaar en vergelijkbaar. Uit de informatieanalyse zijn indicatoren of zelfs KPI's, kritieke prestatie-indicatoren, naar voren gekomen. Daar wordt een organisatie op gestuurd. Dat zijn de feiten!
Feiten zijn over het algemeen numeriek en meestal aggregeerbaar. Voor een verkoopproces kun je denken aan omzet, marge, kosten, aantallen, korting. Dit zijn allemaal numerieke grootheden. Je kunt deze grootheden op verschillende niveaus bekijken. De kwartaalomzet is bijvoorbeeld de som, de aggregatie, van drie maandomzetten. Een maandomzet is op zijn beurt de optelling van alle orders die in dezelfde maand vallen. Feiten worden ook wel meetwaarden (measures) genoemd.
De feitentabel in het sterschema is de centrale tabel in het midden. In figuur 4.1 aan het begin van dit hoofdstuk is het de tabel FactInternetSales. Een feitentabel bestaat uit een verzameling foreign keys en de feiten zelf. De foreign keys (verwijzende/refererende sleutels) verwijzen naar de relevante dimensierecords. In het voorbeeld zijn dat bijvoorbeeld OrderDateKey, ProductKey, CustomerKey, et cetera.

Soorten feiten

Het aggregeerbaar zijn van feiten is een belangrijk aspect. Het hogere management is vaak niet geïnteresseerd in heel gedetailleerde informatie. Zij houden zich meer bezig met de hoofdlijnen en kijken naar bijvoorbeeld omzet op maandniveau. Een teamleider heeft een maandtarget te halen. Om bij te houden of zijn team op schema ligt, houdt hij van dag tot dag de omzet in de gaten. Omzet per dag en omzet tot datum zijn belangrijk. Om de omzet uit te rekenen kun je simpelweg de bedragen van individuele orders bij elkaar optellen. Niet voor alle feiten is het zo gemakkelijk. Er zijn drie soorten feiten:

- additieve feiten;
- niet-additieve feiten;
- semi-additieve feiten.

We zullen deze een voor een bespreken.

Additieve feiten – Additieve feiten zijn getallen die optelbaar zijn. Omzet is een mooi voorbeeld van een additief feit. De omzet van alle individueel verkochte producten kan opgeteld worden tot een totaal omzet. Net zo goed kunnen alle maandomzetten opgeteld worden tot een jaaromzet.



Additieve feiten zijn feiten die over alle dimensies zijn op te tellen.

Niet-additieve feiten – Bij de genoemde voorbeelden van feiten staan ook marge en korting. Stel dat je marge opslaat als percentage van de omzet. Vaak wordt binnen organisaties gesproken over de marge in termen van percentages. De marge op dit product is 15%. Als de marge op een ander product 10% is, is dan de totale marge 25%? Werkte het maar zo eenvoudig. Percentages zijn over het algemeen niet op te tellen. Marge is in dit geval een niet-additieve meetwaarde, ofwel een *non-additive measure*.

Naar mijn mening mag je van de gebruikers van de datamart verwachten dat ze de business kennen en weten wat marge is en dat het een percentage is. Ze moeten daarmee snappen dat ze geen percentages moeten optellen. Verwachtingen zijn wel het begin van alle teleurstellingen en de wet van Murphy is altijd van toepassing: dat wat fout kan gaan, zal ook fout gaan. Met een drag-and-drop-tool die draaitabellen maakt (bijvoorbeeld Excel) worden de cellen (daar waar de feiten in komen te staan) standaard gesommeerd. Excel weet niet dat dat voor een percentage niet de bedoeling is.



Niet-additieve feiten zijn feiten die niet zijn op te tellen, zoals percentages.

In het bovengenoemde voorbeeld is er een eenvoudig alternatief. In plaats van marge als percentage op te slaan, kun je het ook als bedrag opslaan. De marge bedroeg € 10,-. Nu is marge wel optelbaar. Een nadeel van deze oplossing is dat

in de organisatie marge beschouwd wordt als een percentage. Het datawarehousemodel moet zo goed als mogelijk de werkelijkheid beschrijven, het jargon en de definities van de organisatie gebruiken. De juiste keuze is afhankelijk van de tools die gebruikers krijgen en van het niveau van deze gebruikers. Over het algemeen zal marge als optelbaar getal opnemen in het datawarehouse de verstandigste keuze zijn.

Semi-additieve feiten – Stel: een bedrijf heeft als doelstelling geformuleerd dat de marge moet verbeteren door de voorraad terug te dringen zonder dat de levertijd langer wordt. De actuele voorraad moet worden bijgehouden. Gisteren waren er tien stuks van product A op voorraad, vandaag zijn dat er acht. Mag je dus concluderen dat er achttien stuks van product A op voorraad zijn? Werkte dat maar zo met je banksaldo!
Stel dat er gisteren naast product A ook van product B tien stuks op voorraad waren. Waren er dan in totaal twintig stuks op voorraad? Je kunt je afvragen hoe zinvol het is om de voorraad van twee verschillende producten bij elkaar op te tellen, maar het is niet fout. Er zijn twintig stuks goederen op voorraad.
Voorraad is een voorbeeld van een semi-additieve meetwaarde. Over sommige dimensies (bijvoorbeeld de dimensie Product) zijn de getallen wel bij elkaar op te tellen. Over andere dimensies (de dimensie Datum in dit geval) zijn ze niet optelbaar. In de praktijk zal het vaak gaan om de datum/tijddimensie waarover feiten niet aggregeerbaar zijn.

Semi-additieve feiten zijn feiten die over sommige dimensies wel zijn op te tellen, maar over andere dimensies niet.



Soorten feitentabellen

Naast verschillende soorten feiten zijn er ook verschillende soorten feitentabellen. De meeste feitentabellen zijn als het ware registraties van wat er is gebeurd. Bij de supermarkt leidt bijvoorbeeld elk piepje dat je bij de kassa hoort tot een regel in de feitentabel. Het feit van de aankoop is geregistreerd en verandert nooit meer. We spreken hier van een transactionele feitentabel. Maar niet alle feiten zijn zo gemakkelijk te vangen. Andere soorten feitentabellen zijn:

- accumulating snapshot;
- periodiek snapshot.

Accumulating snapshot – Bij een supermarkt pak je wat je nodig hebt en je rekent meteen af. Done deal! Als je een auto gaat kopen, vraag je misschien eerst een offerte op. Het is op zich een feit in het hele verkoopproces dat er een offerte is uitgebracht. Er wordt een regel weggeschreven in de feitentabel. Hopelijk leidt de offerte tot een aankoop. Het al bestaande feitenrecord kan nu worden aangevuld met bijvoorbeeld de aankoopdatum. Er is niet zozeer sprake van een nieuw feit als wel van een veranderde status van een al bestaand feit. Omdat de auto nog rijklaar gemaakt moet worden, is er pas enkele dagen later

sprake van de echte levering. Wederom moet het reeds bestaande feit worden aangepast.
In het bovenstaande geval, namelijk dat feiten worden aangepast aan een veranderende status binnen het totale proces, wordt er gesproken van een *accumulating snapshot* feitentabel. De feitentabel is een weergave, snapshot, van wat je nu weet. Morgen kan het anders zijn.



Een **accumulating snapshot** is een feitentabel die de huidige status van de feiten weergeeft, maar waarbij feiten nog aan verandering onderhevig kunnen zijn.

Periodiek snapshot – Bij de uitleg over semi-additieve meetwaarden hebben we het gehad over voorraad. De registratie van de voorraad is een feit. Het is een feit dat er nu acht stuks op voorraad zijn, net zo goed als dat het een feit is dat dat er gisteren nog tien waren. Om inzicht in de voorraad en het voorraadverloop te krijgen, maak je een periodiek snapshot. Elke dag (of elke maand) bekijk je de voorraad en sla je dat op. Figuur 4.9 is een voorbeeld.

DatumKey	ProductKey	BeginVoorraad	AantalBij	AantalAf	EindVoorraad
20210601	1	10	2	5	7
20210601	2	25	10	15	20
20210701	1	7	3	5	9
20210701	2	20	5	5	20

Figuur 4.9 Periodiek snapshot



Een **periodiek snapshot** is een feitentabel die de stand van de feiten weergeeft op een specifiek moment. Voor een ander moment wordt een ander snapshot gemaakt.

Het mooie van de opzet van het voorbeeld is dat de kolommen AantalAf en AantalBij aggregeerbaar zijn. Dus het aantal producten dat is binnengekomen in een bepaalde periode is gemakkelijk te berekenen. Tegelijkertijd is van elke periode (maand in het voorbeeld) de stand van de voorraad bekend.

Samenvatting

Een feitentabel is een verzameling sleutels die de feiten relateren aan de bijbehorende dimensierecords. Naast deze sleutels bestaat de feitentabel uit allemaal numerieke kolommen, de feiten.
Het soort proces dat beschreven wordt bepaalt het soort feitentabel. Het soort feit bepaalt hoe het feit gebruikt moet/kan worden in rapportages.

4.1.8 Tot slot

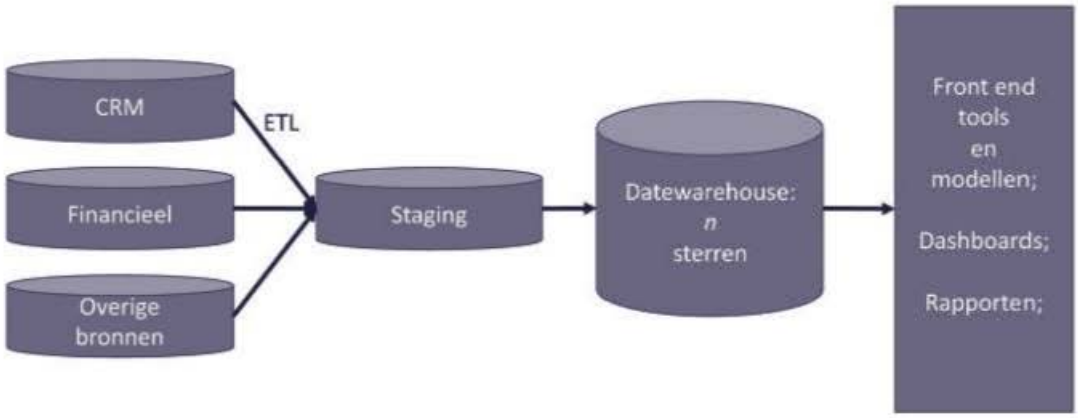
Voorgaande uitleg over dimensioneel modelleren is een uitgebreid verhaal. Tegelijkertijd valt er nog veel meer over te zeggen dan dat je hier hebt gelezen. Als jij degene bent die datamarts gaat ontwikkelen, raad ik je aan *The Data Warehouse Toolkit* van Kimball en Ross te lezen.
Belangrijk binnen de context van dit boek is de rol van het dimensioneel model en vooral die van de datamart. Uit de informatieanalyse volgt wat de organisatie nodig heeft. Wat is het doel van Business Intelligence binnen de organisatie? Welke informatie is er nodig om dat doel te bereiken? Wat voor soort analyses gaat men uitvoeren en met welke KPI's gaat men het proces bijsturen?
In een gestructureerde Business Intelligence-omgeving, een omgeving die in het BI-volwassenheidsmodel zit bij volwassen of hoger, wordt de informatie ontsloten via datamarts. De opzet van de datamart bepaalt wat de gebruikers straks wel en niet kunnen en hoe intuïtief dat is. Daarmee speelt de datamart een belangrijke rol in de Business Intelligence-architectuur.

4.2 Bill Inmon

Bill Inmon wordt door velen gezien als de vader van het datawarehouse. Hij en Ralph Kimball zijn de twee pioniers op dit gebied. Ze zitten echter niet op één lijn als het aankomt op hoe een datawarehouse eruit moet zien. In deze paragraaf zullen we kort ingaan op de kritiek van Inmon op de strategie van Kimball. Ook zullen we kijken naar wat Inmon zelf voorstelt en de kritiek die daar weer op te geven valt. Welke strategie is beter? Dat is aan ieder zelf ter beoordeling, in dit boek spreken we daar geen waardeoordeel over uit, noch een voorkeur.

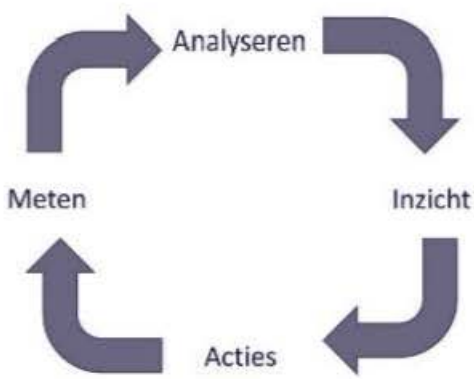
4.2.1 Kritiek op Kimball

Zoals eerder gezegd bestaat in de theorie van Kimball het datawarehouse uit de verzameling van alle sterren. Dimensies worden hergebruikt (conformed dimensies) bij elke feitentabel waarvoor deze relevant zijn. Voordat de gegevens het datawarehouse ingaan, worden ze in een staging area gezet. Een staging-laag biedt een plek om transformaties (de T uit ETL) toe te passen en geeft flexibiliteit aan het laadproces. Een Kimball-architectuur ziet er in hoofdlijnen uit zoals figuur 4.10.



Figuur 4.10 Kimball-architectuur

Bill Inmon is het oneens met de opzet zoals Kimball die voorstaat. Zijn kritiek zit hem voor een groot deel in de Business Intelligence Life Cycle. Figuur 4.11 toont de Life Cycle nogmaals in een iets andere variant. Business Intelligence heeft tot doel om door inzicht, via veranderingen, te komen tot verbeteringen. Die veranderingen zouden goed kunnen inhouden dat de processen zelf anders moeten worden. Anders werken, reorganisaties, op andere kenmerken sturen et cetera.



Figuur 4.11 Business Intelligence Life Cycle

Wat nu als je net, volgens de Kimball-theorie, een datawarehouse hebt gebouwd en je komt tot het inzicht dat je een of meer van de processen anders moet inrichten? Een ster is gemodelleerd naar het proces, dus de bijbehorende ster moet ook worden aangepast. Misschien dat je andere meetwaarden gaat gebruiken om te sturen. Die meetwaarden hebben op hun beurt misschien andere context nodig om zinvol geanalyseerd te kunnen worden. Dimensies moeten dus veranderen. Maar je hebt net veel moeite gedaan om conformed dimensies te bouwen zodat dimensies kunnen worden hergebruikt door meer sterren. Wijzigen van een dimensie heeft dus potentieel gevolgen voor alle sterren in het totale datawarehouse.

De kritiek die veel mensen hebben op de opzet van Kimball is dat het vaak lastig is om het datawarehouse te laten meegroeien met de veranderende wereld. Kleine aanpassingen in de organisatie kunnen grote invloed hebben op het

datawarehouse en daarmee op de hele Business Intelligence. Wat de gebruikers zien, de rapporten en de modellen, is gebaseerd op het datawarehouse. Veranderingen in het datawarehouse hebben daardoor consequenties voor de rapportages.

Veelgehoorde kritiek op de Kimball-aanpak is dat het dimensioneel gemodelleerde datawarehouse slecht in staat is zich aan te passen aan veranderende omstandigheden.

In de hoogste fase van het volwassenheidsmodel moet een bedrijf agile zijn. Dat geldt ook voor het datawarehouse, of beter gezegd voor de technische aspecten van Business Intelligence. Een datawarehouse volledig opgezet volgens de theorie van Ralph Kimball is niet agile.

4.2.2 De Inmon-methodiek

Opzet van het datawarehouse

In grote lijnen zegt Inmon dat een datawarehouse 'gewoon' een genormaliseerde database moet zijn. Met andere woorden, dezelfde ontwerpmethodiek die algemeen aanvaard is voor operationele databases, moet ook worden toegepast voor het datawarehouse.

Het grote verschil tussen normaliseren van databases en het dimensioneel modelleren ervan is het uitgangspunt van wat er gemodelleerd wordt. Bij dimensioneel modelleren wordt voornamelijk het proces gemodelleerd. Denk maar terug aan stap 1 van het dimensioneel modelleren (paragraaf 4.1.2): kies het proces. Het proces staat centraal. Als het proces verandert, verandert het model ook.

Bij normaliseren zijn het de gegevens die gemodelleerd worden. Het gaat er daarbij nog niet om hoe die gegevens gebruikt gaan worden, het is nog geen informatie. Van belang is wat de onderlinge verbanden zijn tussen de gegevens. Een adres bestaat uit een straatnaam, een huisnummer eventueel met toevoeging, een postcode en een plaatsnaam. Het maakt niet uit hoe je werkt of in wat voor business je je bevindt, bovenstaande is altijd waar. Een adres is een adres. Omdat dit soort verbanden onveranderlijk zijn, moeten deze verbanden als uitgangspunt dienen voor het ontwerp van het datawarehouse. Het datawarehouse wordt daarmee veel stabieler door de tijd heen.

Een Inmon-datawarehouse is een genormaliseerde database. Dat betekent dat hij is gemodelleerd op basis van verbanden tussen gegevens die stabiel zijn in de tijd in plaats van op basis van veranderlijke processen.

Een van de tekortkomingen die operationele databases vaak hebben, is het ontbreken van een goede manier om historie bij te houden. De database bevat

alleen de actuele gegevens. In paragraaf 4.1.4 heb je uitgebreid kunnen lezen over Slowly Changing Dimensions, de manier van Kimball om met veranderende dimensieattributen om te gaan. Uiteraard is Inmon zich hier ook van bewust. Maar geschiedenis kan prima genormaliseerd worden.
Omdat in het datawarehouse ook historie wordt opgeslagen, kan de tabelstructuur van het datawarehouse afwijken van die van het bronsysteem. Bovendien worden de gegevens uit meer bronnen gecombineerd, wat tot nog grotere afwijkingen kan leiden.

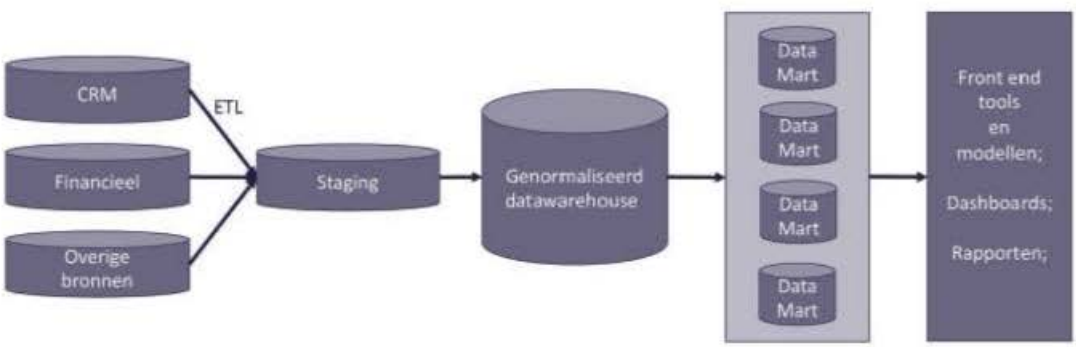
Gebruik van het datawarehouse

Een van de redenen om een datawarehouse te maken, was de complexe tabelstructuur van operationele databases met honderden of meer tabellen. Dit maakt een database lastig om uit te vragen en het is slecht voor de performance. Hoe zit dat dan met het datawarehouse van Inmon?
In de methodiek van Inmon mag niemand aan het datawarehouse komen. Rapporten, dashboards, modellen, ze mogen niet direct uit het datawarehouse lezen. In plaats daarvan is het hele front-end gebaseerd op datamarts. De datamarts worden 'gevoed' vanuit het datawarehouse.



Bij een datawarehouse volgens Inmon wordt het datawarehouse alleen gebruikt om datamarts te vullen, het wordt niet gebruikt als bron voor rapporten of analyses.

Een datamart is een subset van de database gemaakt met een specifiek doel. Meestal is dat doel het ondersteunen van een bepaalde afdeling. Dat vertaalt zich al snel naar een specifiek proces. En om die datamart goed werkbaar, intuitief en snel te maken, is het een dimensioneel gemodelleerde database.



Figuur 4.12 Inmon-architectuur

Gezien dit alles komt een Inmon-architectuur er in grote lijnen uit te zien zoals in figuur 4.12. De hele architectuur, dus alle componenten samen inclusief hier niet getoonde componenten, noemt Inmon de Corporate Information Factory (CIF).

De **Corporate Information Factory** is het geheel van componenten van staging database tot en met datamarts dat ervoor zorgt dat gegevens uit operationele systemen omgevormd worden tot informatie voor de medewerkers van een organisatie.

Als er bij Inmon in de organisatie iets verandert, heeft dat geen impact op het datawarehouse. Het datawarehouse blijft stabiel. In het ergste geval moet de datamart vervangen worden en de rapportages en modellen die daarop gebaseerd zijn ook. Maar dat is logisch verklaarbaar, het proces dat ze beschrijven is immers veranderd, en dus is het acceptabel dat bijbehorende rapporten en databases mee veranderen. Zolang datamarts onafhankelijk zijn van elkaar, blijft de verandering beperkt tot de componenten die met die verandering te maken hebben. Het idee is dat de impact van een kleine verandering klein is, waardoor de totale oplossing beter bestand is tegen de veranderende wereld.

4.2.3 Kritiek op Inmon

Als je de argumenten in deze volgorde leest, denk je misschien dat de Inmon-strategie beter is. Het is niet het doel van dit boek om een waardeoordeel te vellen. Doel is de diverse methodieken op hoofdlijnen te vergelijken, zodat je bekend raakt met de verschillen. Je mag zelf een oordeel vellen. Op de Inmon-methodiek is ook de nodige kritiek gekomen. De kritiek zit hem voornamelijk in de opbouw van een Business Intelligence-project en dan voornamelijk op die van het datawarehouse.
Een ideale opzet voor een Business Intelligence-project is een iteratieve opzet. Korte project cycles, zodat snel resultaten getoond kunnen worden aan de organisatie. Werken volgens de methodiek van scrum, zoals beschreven in hoofdstuk 2, sluit hier goed bij aan. Vroeger duurde Business Intelligence-projecten maanden en pas helemaal aan het einde kon aan de organisatie getoond worden wat de resultaten waren. Tegenwoordig krijg je geen project meer van de grond als je tegen het management moet zeggen: geef mij een paar ton en een jaar de tijd, over een jaar laat ik zien wat ik heb gemaakt. Wat organisaties willen is met een klein budget en korte doorlooptijd snel meerwaarde zien. Als die meerwaarde er daadwerkelijk is, kan de volgende fase van het project gestart worden. Als er geen meerwaarde is, gaan we ook niet verder.
Wat heeft dit verhaal te maken met de Inmon-methodiek? Om dat uit te leggen gaan we eerst even terug naar Kimball. Stap 1 in de Kimball-methodiek van modelleren was het kiezen van het te modelleren proces. Je ontwerpt de bijbehorende sterstructuur en zodra die af is, kunnen er rapporten gemaakt gaan worden. Dat zijn de '*deliverables*' van het project. Je kunt aan de organisatie laten zien wat het project oplevert. Met een beetje geluk worden gebruikers zo enthousiast dat ze zelf gaan vragen om meer, om dezelfde soort informatie met betrekking tot andere processen. Deze manier van werken sluit prima aan bij de iteratieve projectopzet die tegenwoordig gewenst is.



De Kimball-methodiek maakt een iteratieve projectaanpak met kleine stappen en snel opleveren van resultaten mogelijk.

Nu terug naar de Inmon-methodiek. Het genormaliseerde datawarehouse omvat eigenlijk alles, gemodelleerd naar de gegevens. Alleen valt het 'alles' niet zo goed te rijmen met de iteratieve projectopzet. Als je slechts de gegevens van bijvoorbeeld het verkoopproces opneemt in het datawarehouse en je wilt later het inkoopproces ook opnemen, kan het zijn dat er structuurveranderingen nodig zijn in het datawarehouse. Maar die hebben impact op de datamarts die al eerder zijn gemaakt.
Bij Kimball is hetzelfde eigenlijk ook waar. De impact is afhankelijk van hoe goed je in staat bent geweest om conformed dimensies te maken. Het probleem is bij een Inmon-aanpak over het algemeen groter, wat je meer dwingt richting een 'big bang'-implementatie.



De Inmon-opzet van een datawarehouse dwingt je eerst om het hele datawarehouse te ontwerpen. Dit is moeilijk te rijmen met de tegenwoordig geprevaleerde iteratieve manier van werken.

4.2.4 *Samenvatting*

Samengevat kun je zeggen dat de Kimball-methode gemakkelijker is in de uitvoering van het project. Het is eenvoudig om een kleine scope te kiezen. Daardoor kun je snel resultaten opleveren. Bovendien vergroot je de kans op succes. Het nadeel is dat het moeilijk kan zijn om het datawarehouse mee te laten groeien met veranderende eisen en wensen van de eindgebruikers.
De Inmon-methode levert een in de tijd gezien stabieler datawarehouse op. Een bedrijf kan langer profiteren van zijn investering. De grote uitdaging zit in het bepalen van de scope en het klein houden van je projecten.
Veel mensen hebben over de jaren heen mengvormen proberen te bedenken tussen de iteratieve projectaanpak van Kimball en het stabiele datawarehouse van Inmon. De laatste jaren is Data Vault steeds populairder geworden (in Nederland). Data Vault probeert ook het beste van beide aanpakken te combineren. Inmon heeft zich ondertussen geschaard achter de theorie van Data Vault. Dat is dan ook het onderwerp van de volgende paragraaf.

4.3 Data Vault

4.3.1 Definitie

Zoals gezegd is Data Vault een opzet van een datawarehouse die probeert om het beste van de Kimball- en de Inmon-methodiek te combineren. De Data Vault is ontwikkeld door Dan Linstedt. Zelf geeft hij op zijn site (danlinstedt.com/about/data-vault-basics) de volgende definitie:



'The Data Vault is a detail oriented, historical tracking and uniquely linked set of normalized tables that support one or more functional areas of the business. It is a hybrid approach encompassing the best of breed between 3rd normal form and star schema.'

Vrij vertaald staat daar dat een Data Vault bestaat uit genormaliseerde tabellen die gedetailleerde informatie over een of meer deelgebieden van een organisatie bevatten. De tweede zin zegt expliciet dat een Data Vault het beste van zowel normaliseren als van sterschema's bevat.



Data Vault is een verzameling gekoppelde tabellen die gedetailleerde en historische informatie bevat van een of meer processen. Het is een mix tussen normaliseren en dimensioneel modelleren.

De tabelstructuur die Linstedt voorstaat is heel generiek. Een Data Vault is gemakkelijk aan te passen aan nieuwe en veranderde eisen. Het is ook gemakkelijk uit te breiden met nieuwe functionele gebieden. Deze combinatie maakt het mogelijk om een Data Vault-structuur op te zetten voor één specifiek proces binnen een organisatie. Later kan deze structuur uitgebreid worden met de gegevens die nodig zijn voor andere processen. Uitgebreid worden is hier iets anders dan aangepast worden. De uitbreidingen hebben geen effect op wat al bestaat.

De Data Vault-structuur levert nog veel meer tabellen op dan het aantal tabellen dat ontstaat bij normaliseren. Gebruikers rechtstreeks laten werken op een Data Vault-structuur is dan ook een slecht idee, zowel qua intuïtiviteit als qua performance. Linstedt stelt dan ook, net als Inmon, voor om het datawarehouse te ontsluiten via een verzameling dimensioneel gemodelleerde datamarts.

4.3.2 Hubs, Links and Satellites

Net zoals bij dimensioneel modelleren is er een stappenplan om tot een Data Vault-structuur te komen. De stappen zijn wel redelijk technisch georiënteerd:

1. Bepaal de business keys, de hubs.
2. Bepaal de relaties tussen business keys, de links.
3. Bepaal de beschrijvende informatie van de business keys, de satellites.
4. Voeg onafhankelijke tabellen toe, zoals een datumtabel en code/beschrijving-tabellen.
5. Voeg, indien nodig, tabellen toe om performance van query's te ondersteunen.

Laten we elke stap kort bekijken.

Hubs

De hubs zijn eigenlijk waar het in een organisatie (binnen een proces) over gaat. Hubs zijn het beste te vergelijken met dimensietabellen uit de datamart, maar met twee grote verschillen. Ten eerste bevat een hub-tabel geen beschrijvende informatie maar alleen keys. Ten tweede zijn hubs genormaliseerde tabellen en geen platgeslagen tabellen.



Hubs zijn tabellen die vergelijkbaar zijn met dimensietabellen, maar dan zonder alle beschrijvende attributen.

Bij een verkoopproces heb je de hubs order, product, klant, factuur, et cetera. In het stappenplan wordt gezegd: bepaal de business keys. Gedacht vanuit de operationele database die het verkoopproces ondersteunt, zijn er sleutels (keys) voor deze entiteiten. Waarschijnlijk is er sprake van een uniek productnummer. Beter nog is om een logische key te gebruiken zoals een unieke productnaam als die voorhanden is. Net zo goed bestaan er factuurnummers, klantnummers en ordernummers. Een hub bestaat uit de volgende kolommen:

- Surrogate key, unieke datawarehouse key.
- Business key, de logische sleutel in de bron (dat is niet per se hetzelfde als de primaire sleutel van de bron).
- Load date, de datum dat een record in het datawarehouse is opgenomen.
- Record source, beschrijvend veld dat aangeeft uit welke bron de business key komt.
- Eventueel een Load audit ID, dat verwijst naar een record met metadata over het laadproces, zoals packagenaam en -versie en laaddatum.

Het opvallendste is dat een hub geen andere velden bevat. Dus de hub Product heeft geen velden voor naam, prijs, gewicht, kleur, et cetera. Dat soort velden komen terecht in de satellites.

Links

Een proces wordt gekenmerkt door de interactie die de verschillende entiteiten (de hubs) met elkaar hebben. Deze interactie wordt gevangen in linktabellen. De interacties tussen links worden ook wel transacties of associaties genoemd.

Links zijn tabellen die vergelijkbaar zijn met feitentabellen. Ze beschrijven de relaties (associaties) die verschillende hubs onderling hebben. Ze bevatten de feiten zelf niet.



Een klant ontvangt een factuur. Klant en factuur hebben dus iets met elkaar te maken. Beide zijn hubs, want er is een business key. Of in minder technische terminologie: het zijn primaire entiteiten in het businessproces. De relatie wordt gevangen in een linktabel. Een link bevat de volgende kolommen:

- Surrogate key, unieke datawarehouse key.
- De keys van de hubs, dus hub_klant_key en hub_factuur_key in bovenstaand voorbeeld. Dit zijn foreign keys naar de hubs.
- Load date, de datum dat een record in het datawarehouse is opgenomen.
- Record source, beschrijvend veld dat aangeeft uit welke bron de business key komt.
- Eventueel een Load audit ID, dat verwijst naar een record met metadata over het laadproces zoals packagenaam en -versie en laaddatum.

Linktabellen kunnen met feitentabellen worden vergeleken in de zin dat ze de grain bepalen. Meer keys naar hubs opnemen in een link verandert de grain. Voor links geldt net als voor hubs dat er geen beschrijvende velden in zitten. Een link bevat slechts de hierboven benoemde kolommen.

Satellites

De satellites bevatten uiteindelijk alle beschrijvende kolommen. Dus informatie als kleur en prijs van een product komen in een satellite die een relatie (foreign key) heeft met de hub product. Factuurbedrag is een kolom in een satellite die hoort bij de hub factuur.

Satellites zijn de tabellen die de feiten en de dimensieattributen bevatten. Ze zijn door middel van foreign keys gekoppeld aan links of hubs.

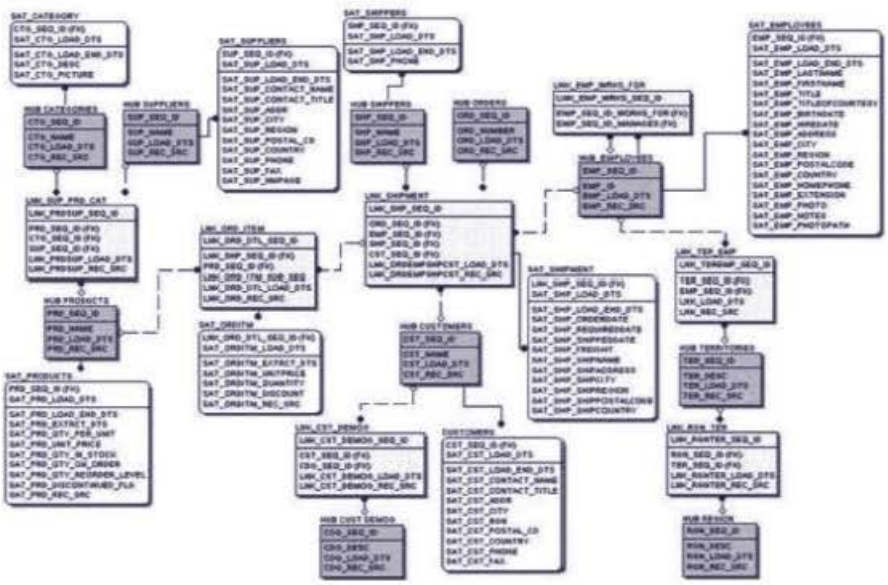


Een hub of een link kan gekoppeld zijn aan meer satellites. Je kunt attributen die uit verschillende bronsystemen komen, opnemen in aparte satellites. Maar ook de frequentie waarmee attributen door de tijd heen kunnen veranderen kan een reden zijn om ze samen (of juist niet) in een satellite op te nemen. Een satellite bestaat uit de volgende kolommen:

- De key van de hub of de link waar deze satellite bij hoort. Dit is een foreign key.
- Load date, de datum dat een record in het datawarehouse is opgenomen.

- End date, de datum dat een nieuwere versie van het record in het datawarehouse wordt opgenomen. Dit is vergelijkbaar met een SCD type 2 zoals eerder beschreven.
- Record source, beschrijvend veld dat aangeeft uit welke bron de business key komt.
- Eventueel een Load audit ID, dat verwijst naar een record met metadata over het laadproces zoals packagenaam en -versie en laaddatum.
- De 'echte' beschrijvende velden waar het om te doen is.

In figuur 4.13 zie je een voorbeeld van een Data Vault-model zoals het staat op de site van Linstedt. Dit model is gebaseerd op de (oude) voorbeelddatabase van Microsoft, Northwind. Je kunt Northwind nog altijd downloaden van de Microsoftsite. Op de site van Linstedt vind je ook de oorspronkelijke tabelstructuur van Northwind.



Figuur 4.13 Data Vault-structuur

4.3.3 Voordelen Data Vault

Dit boek is geen promotieboek voor Data Vault. Voor een uitgebreide lijst met (geclaimde) voordelen verwijs ik naar de site van Dan Linstedt. De volgende punten die je op de site vindt, zullen we hier kort bespreken:

- eenvoud van laden;
- snelheid van laden;
- betrouwbaarheid van laden;
- compliance.

Eenvoud van laden

De tabelopbouw is in een Data Vault heel generiek. Dit maakt dat het gemakkelijk wordt om een Data Vault te vullen met gegevens vanuit de operationele databases.

Bij een stermodel of een genormaliseerde database is elke tabel anders. Voor elke tabel moet er goed over nagedacht worden hoe deze te vullen. De query's die nodig zijn zullen elke keer anders zijn.
In de vorige paragraaf is van elk soort tabel beschreven uit welke kolommen die bestaat. Dat is elke keer hetzelfde. Het is relatief gemakkelijk om een template te maken voor het laden van een hub, een link of een satellite, omdat ze zo generiek zijn. Een goede template maakt het ontwikkelen van ETL-processen gemakkelijk en snel. Veelal is het automatisch genereren van de ETL zelfs mogelijk. Dat maakt doorlooptijden van projecten korter en verlaagt dus de kosten.

Snelheid van laden

Alle hubs zijn onafhankelijk van elkaar. Dat geldt ook voor links en satellites. De enige afhankelijkheid tussen tabellen zit in de foreign keys die zijn opgenomen in de links en satellites. Maar keys in Data Vault kunnen berekend worden vanuit de business keys (Data Vault gebruikt hash keys). Dat houdt in dat alle tabellen onafhankelijk van elkaar geladen kunnen worden. In grotere systemen houdt dat in dat meer tabellen tegelijkertijd geladen kunnen worden zonder dat rekening gehouden hoeft te worden met de volgorde waarin de tabellen geladen worden.
Deze constatering betekent dat er heel veel zaken parallel geladen kunnen worden. Met het huidige aantal cores dat beschikbaar is in servers, kan dat tot heel goede laadprestaties leiden.

Betrouwbaarheid van laden

Data Vault heeft als uitgangspunt dat er geen enkele verandering wordt gedaan op de gegevens. Bedrijfsregels zoals berekeningen, aanvullen van incomplete gegevens en verbeteringen van fouten moeten gedaan worden in de datamart. Dat maakt het laden betrouwbaarder en het loggen van fouten eenvoudiger. Dat moet leiden tot een betrouwbaarder laadproces.

Compliance

Door bovenstaande en door de opzet met source keys en verwijzingen naar auditrecords, is het gemakkelijk om te voldoen aan verschillende complianceregels.

4.3.4 Samenvatting

Van oorsprong zijn er twee grote datawarehousestromingen: de aanhangers van de Kimball-methodiek en die van de Inmon-methodiek. In de praktijk zijn mensen met beide methodieken tegen beperkingen aangelopen.
Data Vault poogt met een generieke tabelstructuur het beste van de Kimball- en de Inmon-strategie te combineren. Bovendien speelt het in op de krachtiger

hardware die we tegenwoordig hebben. Ook houdt het meer rekening met strengere complianceregels van de laatste jaren.

4.4 Kimball, Inmon en Data Vault vergeleken

In paragraaf 4.2 hebben we de Kimball- en de Inmon-methodologie vergeleken. Die paragraaf eindigde met de verwachting dat Data Vault het beste van beide pakt. Ook nu geldt weer dat we in dit boek geen waardeoordeel vellen over de verschillende besproken strategieën. Het is goed om ze allemaal op hoofdlijnen te kennen en te begrijpen.

De argumenten die we gebruikten om Kimball en Inmon te vergelijken waren:
- stabiliteit van het datawarehouse;
- gemakkelijk iteratief kunnen ontwikkelen.

Stabiel (resilient)

Doordat Data Vault business keys als basis neemt (de hubs) zonder alle bijbehorende rompslomp, ontstaat een stabiel systeem. Als er nieuwe kenmerken ontstaan in de loop van de tijd, of als er nieuwe indicatoren (KPI's) worden bedacht binnen de organisatie, kunnen er simpelweg nieuwe satellites worden bijgemaakt. De bestaande tabellen blijven exact hetzelfde. Query's op de Data Vault hebben dus geen last van de wijzigingen. Maar ook het laadproces heeft er geen last van. Er moet alleen nieuwe code geschreven worden om de nieuwe tabel te vullen.

Veranderingen in de processen hebben geen invloed op de hubs en de bijbehorende satellites. Misschien dat er nieuwe links (met hun eigen satellites) gemaakt moeten worden.

Bovenstaande houdt in dat het Data Vault-model dezelfde (of zelfs een grotere) stabiliteit heeft dan een gewoon genormaliseerd datawarehouse. Dat is wat Inmon wel heeft en Kimball niet.

Iteratief

De Data Vault-structuur is heel generiek. Het is te allen tijde mogelijk om het model uit te breiden met nieuwe hubs en links. Dat komt overeen met nieuwe processen toevoegen aan het datawarehouse. Het eenvoudigste Data Vault-model bestaat uit een hub met een satellite. Voor het modelleren van een proces zullen er enkele hubs en links bestaan, elk met hun eigen satellites. Als je het datawarehouse wilt uitbreiden met nieuwe processen, gebruik je de hubs die er al zijn, eventueel aangevuld met nieuwe satellites. Je kunt altijd nieuwe hubs introduceren. Het nieuwe proces komt tot uitdrukking in nieuwe links.

Het voorgaande houdt in dat het gemakkelijk is iteratief te ontwikkelen. Je kunt klein beginnen en dus met een beperkt budget beginnen. Hiermee kun je ook snel resultaten aan de organisatie laten zien. Van daaruit kun je het

datawarehouse en daarmee de totale Business Intelligence-oplossing uitbouwen. Dit is wat Kimball wel heeft en Inmon niet.

4.5 Tot slot

In organisaties die in volwassenheidsmodellen hoog scoren, is het datawarehouse de centrale plek om alle relevante gegevens, ongeacht de herkomst, op te slaan. Vanaf hier gaan de gegevens naar de gebruikers. Bij een Kimball-implementatie wordt het datawarehouse zelf gebruikt. Bij Inmon en Data Vault zit er nog een laag van datamarts tussen het datawarehouse en de gebruikers ervan. Het datawarehouse vormt daarmee het fundament van je Business Intelligence-oplossing. Als je meer wilt lezen over het ontwerpen van databases in het algemeen en datawarehouses in het bijzonder, kun je het boek *Database modelleren* (Ter Braake, 2020) lezen.
De introductie van Big Data in combinatie met de cloud heeft de inzichten op het gebied van datawarehousing doen veranderen. Zoals gezegd wordt tegenwoordig vaak een Data Lake geïmplementeerd. In het volgende hoofdstuk leer je wat een Data Lake is en waarom een Data Lake interessant is om te gebruiken.

Op de website bij dit boek kun je terecht voor de oefenvragen bij hoofdstuk 4.





Big Data en Data Lakes

5

Datawarehousing zoals beschreven in hoofdstuk 4 is de klassieke manier om Business Intelligence te implementeren. In die klassieke manier worden (voornamelijk) eigen relationele databases ontsloten via datawarehouses en andere BI-tools. De bronnen die we besproken hebben zijn veelal CRM-systemen, ERP-systemen en andere transactie-georiënteerde systemen. Dat is tevens het soort systeem waar relationele database-platformen zoals SQL Server en Oracle excelleren. Het zijn ook de systemen waar we ons eind jaren tachtig/begin jaren negentig van de vorige eeuw voornamelijk mee bezig hielden. Maar slechts een fractie van de digitale gegevens die wereldwijd bestaan, zit in relationele databases. We gebruiken in klassieke BI dus meer data (en dus 'potentiële' informatie) niet dan wel. Met Big Data gaan we ook veel gegevens die niet in relationele databases zitten wel gebruiken. Je kunt dan denken aan sensordata, logfiles, ongestructureerde teksten, foto's, gegevens in zogenaamde noSQL-databases en ga zo maar door. Eigenlijk alle digitaal opgeslagen data bevat informatie. Met Big Data willen we die informatie ontsluiten. Met Big Data komt ook het Data Lake om de hoek kijken. Voordat we het over Data Lakes gaan hebben, is het goed om eerst precies te weten wat Big Data nou eigenlijk behelst.

5.1 Big Data

Het vakgebied Business Intelligence is niet wezenlijk veranderd sinds de introductie van Big Data. Ook Big Data verandert niet waar Business Intelligence om draait: het nemen van door informatie gefundeerde beslissingen. Maar er is in de wereld wel veel veranderd sinds de introductie van Business Intelligence. Het internet is groot geworden en daarmee is de invloed van sociale media groot geworden. Ook heeft het internet IoT (Internet of Things) mogelijk gemaakt waar apparaten op grote schaal gegevens genereren en naar databases sturen. De hardware die we tot onze beschikking hebben is veel krachtiger geworden en tegelijkertijd veel goedkoper. En met de cloud is er bijna geen limiet meer op hoe schaalbaar applicaties en databases zijn. We hebben, kortom, nieuwe soorten digitale gegevens gekregen. We hebben ontzettend veel meer digitale gegevens tot onze beschikking en we willen andere soorten vragen stellen dan voorheen. Om andere vragen te beantwoorden met andere soorten gegevens

moet je wellicht innoveren ten opzichte van klassieke oplossingen. Als dit voor jou waar is, heb je te maken met Big Data.



Big Data betekent dat het soort gegevens dat je hebt in combinatie met wat je ermee wilt doen, je dwingt om te innoveren.

Heel flauw gezegd betekent dit ook dat als je grootste gegevensbestand een Excel-sheet is, en je in de toekomst meer dan een miljoen rijen wilt gaan opslaan, je misschien wel moet gaan werken met MS Access of zelfs SQL Server. Uiteraard is switchen van Excel naar Access geen Big Data-project. Het gaat echter om het idee. Zoals gezegd hebben we nieuwe soorten gegevens gekregen, hebben we meer gegevens gekregen en stellen we andere soorten vragen. We hebben ook nieuwe software gekregen en ook de mogelijkheden van de hardware zijn gegroeid. Big Data is een verzamelnaam voor die nieuwe soort gegevens, Advanced Analytics is het stellen van die nieuwe vragen.
De nieuwe soort gegevens heeft vier (hoofd)kenmerken. Deze vormen samen de definitie van Big Data. Big Data wordt vaak gedefinieerd als de 4 V's (gartner.com).



Big Data wordt gedefinieerd door wat vaak de 4 V's wordt genoemd:
- Volume
- Variety
- Velocity
- Variability

In de volgende pargrafen worden deze vier kenmerken verder besproken.

5.1.1 Volume

In mijn ogen staat het verkeerde kenmerk bovenaan en lijkt het vaak alsof het verkeerde kenmerk ook de naamgever is. Big in Big Data lijkt sterk te suggereren dat we te maken hebben met heel veel data. Het suggereert dat het volume van de gegevens heel groot is. De hoeveelheid gegevens die je hebt, het volume, is niet het typerende kenmerk van Big Data, maar is zeker een van de kenmerken. De meeste relationele databases draaien op één server. Als die server te klein wordt voor wat je ermee wilt doen, moet je een grotere server kopen. Dat noemen we scale-up. Vertical scaling is een andere term die hier wordt gebruikt. Maar wat als je al de grootste server hebt die er bestaat?



Scale-up is het inzetten van een grotere server.

Nog niet zo heel lang geleden vonden we een database van een terabyte groot. Nog altijd zijn de meeste databases kleiner dan dat. Tegelijkertijd hebben we het tegenwoordig regelmatig over petabyte (PB) of zelfs exabyte (EB). Een petabyte is duizend terabyte, een miljoen gigabyte. Een exabyte is duizend petabyte. Gelukkig is ook de hardware een stuk krachtiger en betaalbaarder geworden zodat je met databases van deze grootte ook echt kunt werken. Toch loop je met scale-up onvermijdelijk een keer tegen de limiet aan.
In plaats van een grotere server kopen kun je ook meer servers inzetten en het werk verdelen over die servers. Je maakt dan een zogenaamd cluster van servers. Dan doe je niet een scale-up, maar een scale-out (ook wel horizontal scaling genoemd). Scale-out is in theorie oneindig schaalbaar. Je kunt er altijd weer meer servers bijplaatsen.

Scale-out is het verdelen van de workload over een cluster van meerdere servers.



Tegenwoordig zijn er verschillende relationele databaseproducten op de markt die gebruikmaken van scale-out. Voorbeelden zijn Exadata van Oracle, Netezza van IBM, Teradata en Azure Synapse Analytics. Met deze producten kun je databases maken die vele tientallen terabytes beslaan. Afhankelijk van het product is er zelfs geen echte bovengrens meer.

Tot volumes van petabytes aan toe kunnen we dus oplossingen bouwen met relationele databases. Daarvoor is de theorie van de voorgaande hoofdstukken dus gewoon van toepassing. Uiteraard verg je dan het uiterste van je hardware en software, wat wel kosten met zich meebrengt. De discussie moet zich niet alleen richten op het opslaan. Je moet ook nog met de gegevens kunnen werken met acceptabele performance. Bij heel grote volumes kom je met het verwerken van je gegevens een keer in de problemen. Wellicht dient een ander probleem zich echter eerst aan: een relationele database slaat gegevens op in tabellen en gaat dus uit van heel gestructureerde gegevens. Dat brengt ons op de tweede V: variety.

5.1.2 Variety

In het kader van variety kunnen we gegevens onderverdelen in drie soorten. Dat zijn:

- gestructureerde gegevens (Structured Data);
- semi-gestructureerde gegevens (Semi-structured Data);
- ongestructureerde gegevens (Unstructured Data).

Gestructureerde gegevens

Bij gestructureerde gegevens is er een schema, een datamodel, bekend. Met het schema worden de metadata bedoeld: de beschrijving van alle kolommen, de datatypes van de kolommen en het domein (de mogelijke waarden) van de kolommen. Vaak is er sprake van een sterke koppeling tussen de gegevens zelf en de metadata. Een tabel in een database bevat de kolomdefinities, de metadata en de gegevens zelf.

Er wordt ook vaak gesproken over schema-on-write. Voordat je een record kunt wegschrijven in een database, moet er al een tabel zijn. De metadata is er dus eerder dan de gewone data.



Met **schema-on-write** bedoelen we dat de metadata al moet bestaan voordat de gewone gegevens opgeslagen kunnen worden.

Relationele databases zijn heel erg goed in dit soort gegevens opslaan en verwerken. Veel transactionele systemen, zoals financiële systemen, leveren voornamelijk dit soort sterk gestructureerde gegevens op. Relationele databases zijn daarmee voorlopig nog niet weg te denken uit de BI-wereld.

Semi-gestructureerde gegevens

Bij semi-gestructureerde gegevens is er nog wel sprake van enige metadata maar niet meer zo in beton gegoten als bij gestructureerde gegevens. Je kunt bijvoorbeeld denken aan gegevens in de vorm van XML of JSON. Waarschijnlijk is een beschrijving van de documentstructuur, de metadata, wel ergens beschikbaar. Maar dat is al niet noodzakelijk. Bovendien zit er veel meer vrijheid in deze structuur dan dat er vrijheid zit in een tabel.

Zaken als e-mails, tweets en Facebookposts zijn ook semi-gestructureerd. Je weet altijd wel iets als afzender en tijdstip. Je weet dat er een bericht is. Maar het bericht zelf is gewoon tekst en dat kan van alles zijn. De opsteller heeft volledige vrijheid, dat is eigenlijk alles wat je weet.

Vaak vallen logbestanden ook onder de semi-gestructureerde gegevens. Het zijn grote bestanden met meldingen, vaak opgeslagen als CSV (Comma Separated Values)-bestanden. Denk bijvoorbeeld aan de logbestanden van applicaties met allemaal meldingen en foutmeldingen ten aanzien van de applicatie. Vaak is er wel structuur met iets als een timestamp, errorcode en beschrijving, maar dat wordt niet afgedwongen in het logbestand zelf. We gebruiken wat we weten van de gegevens pas als we de gegevens gaan verwerken. We noemen dat soms wel schema-on-read.



Met **schema-on-read** bedoelen we dat we de metadata die gegevens beschrijft pas gebruiken bij het verwerken van de gegevens, niet bij het schrijven (registreren) van die gegevens.

Ongestructureerde gegevens

Ten slotte heb je de ongestructureerde gegevens. Helemaal ongestructureerd is het nooit. Als je een bestandsformaat weet, heb je al kennis over het soort informatie.

Bij ongestructureerde data kun je denken aan bijvoorbeeld foto's. Als je een foto hebt in jpg-formaat, dan weet een grafische expert wellicht vrij veel over pixels, kleur, helderheid en andere zaken. Voor de meesten van ons is de inhoud niets-zeggend tenzij je het bestand opent in een foto-app.

Zoals eerder al genoemd kan een foto heel veel informatie bevatten. Als je bijvoorbeeld een foto maakt van een demonstratie, kun je zien hoeveel mensen er zijn en wat voor soort mensen (leeftijd, geslacht). Als de gezichten herkenbaar zijn, kun je wellicht achterhalen wie er op de vorige demonstratie ook waren. Ook zou je aan de gezichtsuitdrukking de stemming kunnen afleiden en zo in de gaten kunnen houden of de stemming aan het omslaan is. In stadions zijn dit soort toepassingen allang in gebruik en ook in het openbare leven komt dit steeds meer voor.

Van de totale hoeveelheid digitale gegevens die er wereldwijd bestaan, is maar een klein deel gestructureerd. Een groot deel is dus on- of semi-gestructureerd. Deze gegevens zijn, zeker in grote aantallen, lastig te verwerken met relationele databases. Big Data-oplossingen zijn wel in staat deze gegevens om te zetten in zinvolle informatie. Informatie waarop wij beslissingen kunnen nemen. Dus worden onze BI-oplossingen beter met Big Data-implementaties. Big Data is daarmee aanvullend op klassieke BI, niet ter vervanging ervan.

Big Data is aanvullend op klassieke BI. Het is geen vervanging van klassieke BI.



5.1.3 Velocity

Het derde kenmerk van Big Data is velocity, ofwel snelheid. Dit kan twee dingen betekenen: de snelheid waarmee gegevens ontstaan of de snelheid waarmee we de gegevens moeten verwerken tot bruikbare informatie. Hier komt dus het al dan niet realtime zijn van databases om de hoek kijken.

Met streamingdata-applicaties kunnen grote hoeveelheden nieuwe data verwerkt en/of opgeslagen worden. Je moet dan bijvoorbeeld denken aan IoT (Internet of Things)-oplossingen waarbij veel nieuwe data gegenereerd wordt en waar in sommige situaties (bijna) meteen op die nieuwe data gereageerd moet worden. Relationele databases hebben een zekere overhead als er gegevens worden weggeschreven. Daar waar heel grote hoeveelheden nieuwe gegevens ontstaan en opgeslagen moeten worden, kan het zijn dat je de limieten bereikt van wat een relationele database aankan.

Voor alle drie tot nu toe genoemde kenmerken geldt dat ze alleen niet snel een issue zullen vormen. Een database, mits goed ingericht, kan heel veel data opslaan en verwerken, kan semi-gestructureerde gegevens opslaan en

verwerken en kan ongestructureerde gegevens opslaan. Ook kan een database een grote insert-rate aan. Daar waar je deze kenmerken gaat combineren, ga je tegen grenzen aanlopen. Dan moet je gaan innoveren, ofwel oplossingen zoeken anders dan met relationele databases mogelijk is.

5.1.4 Variability

Het vierde belangrijke kenmerk voor Big Data is variability of veranderlijkheid. Denk bijvoorbeeld aan productinformatie voor een webwinkel. Een webwinkel zou bijvoorbeeld televisies kunnen verkopen en kenmerken opslaan zoals schermdiagonaal, scherpte en verversingssnelheid. Later in de tijd gaat dezelfde webwinkel ook wasmachines verkopen. Nu moeten er kenmerken als laadgewicht en toerental bijgehouden worden. Elk soort product heeft zijn eigen kenmerken. Bovendien zullen er in de toekomst ongetwijfeld nieuwe producten aan het assortiment worden toegevoegd met weer nieuwe kenmerken. Een tabel met vooraf gedefinieerde kolommen is te star in dit soort situaties. Er zijn best ontwerpen te bedenken zodat een relationele database toch kan omgaan met het feit dat elke rij eigenlijk andere kolommen nodig heeft. Maar die hebben allemaal nadelen die erger worden bij grotere datavolumes of meer gebruik van de gegevens. Veel webwinkels gebruiken dan ook een noSQL-database om hun productinformatie in op te slaan.
NoSQL-databases zijn over het algemeen flexibeler in hoe gegevens worden opgeslagen en zijn meestal toegerust op de schaalbaarheid die hoort bij de V van volume. Onze BI-oplossing gebruikt deze database als bron en moet de flexibiliteit dus ook hebben. Een conformed productdimensie maken zoals beschreven in het vorige hoofdstuk is bijna onmogelijk.

5.1.5 Andere kenmerken

Sommige mensen nemen niet de 4 V's van Big Data als definitie, maar komen met 5 of met 6 V's. De kenmerken die er dan bijkomen zijn:

- Veracity
- Value

Veracity

Mogelijke vertalingen voor veracity zijn waarheid, echtheid en geloofwaardigheid. In Big Data zegt het iets over de kwaliteit en betrouwbaarheid en over de hoeveelheid ruis in de gegevens. Kwaliteit en betrouwbaarheid spelen met name een rol bij gegevens die je extern krijgt. In dit boek heb ik bijvoorbeeld enkele definities overgenomen van Wikipedia. Maar hoe betrouwbaar is Wikipedia en daarmee de overgenomen definities? En als de input niet betrouwbaar is, wat zegt dat dan over de resultaten van de analyses die je uitvoert op die input?

Met ruis bedoelen we data (bytes) die niet tot informatie leiden. Een tabel in een database heeft bijna helemaal geen ruis. Elke byte bevat informatie. Een foto van 150 KB waaruit je alleen herleidt dat er drie mensen op staan die jonger dan tien jaar zijn, heeft heel veel ruis. Er zijn meer bytes die geen informatie bevatten dan bytes die dat wel doen. De signal-to-noise-ratio is heel laag. Wegen bij een lage signal-to-noise-ratio de kosten voor opslag en verwerking nog wel op tegen de meerwaarde die de informatie biedt?

De **signal-to-noise-ratio** is de verhouding tussen informatie en ruis in data.



Enkele jaren geleden wogen de kosten van het analyseren van foto's niet op tegen de opbrengst. Met de toegenomen kracht van hardware en software (en met de introductie van de cloud met zijn bijna oneindige resources) is dat nu vaak niet meer waar. De grens schuift op.

Value

Met value wordt de meerwaarde van gegevens bedoeld, de bruikbaarheid. Vaak worden in artikelen en boeken over Big Data sociale media zoals Twitter gebruikt als voorbeeld. Maar hoe bruikbaar is Twitter eigenlijk? Het antwoord hangt natuurlijk volledig af van wat je wilt. Welke vraag moet je beantwoorden? De waarde van Twitter is een stuk groter als je moet weten wat de publieke opinie over een grote Hollywoodfilm is dan wanneer je wilt weten of een hotel dat je wilt boeken goed is.

De waarde wordt naast de te beantwoorden vraag bepaald door vragen als:

- Is de data volledig?
- Wat is de veracity?

5.1.6 Samenvatting

Zoals gezegd is Business Intelligence niet veranderd door Big Data. Wat is veranderd is dat we andere soorten gegevens hebben en dat vereist andere manieren om die gegevens op te slaan. Daarnaast zijn bewerkingen zoals gezichtsherkenning en emotiedetectie, image recognition, tekst mining en andere aan machine learning en artificial intelligence gerelateerde data-analyses door betere en goedkopere hardware en door de cloud, gemeengoed geworden. Een paar jaar geleden waren dit soort zaken nog aanzienlijk minder ver ontwikkeld en alleen bereikbaar voor bedrijven met heel grote budgetten.

Door de nieuwe verwachtingen en andere gegevens hebben we nieuwe andere tools nodig om de gegevens te bewerken en andere strategieën om het maximale uit onze Business Intelligence-investeringen te halen. Dat heeft geleid tot het ontstaan van Data Lakes, het onderwerp van de volgende paragraaf.

5.2 Data Lake

Laten we beginnen met te kijken naar enkele beperkingen die relationele datawarehouses zoals in hoofdstuk 4 besproken met zich meebrengen. Daarna kunnen we inzoomen op wat een Data Lake is.

5.2.1 Gevolgen voor Business Intelligence

Door zowel de opkomst van Big Data als door de opkomst van Advanced Analytics, machine learning and artificial intelligence kwamen een aantal beperkingen van datawarehouses aan het licht. Deze tekortkomingen zijn:

- Er worden voornamelijk (alleen?) relationele bronnen ontsloten.
- Het is moeilijk om realtime datawarehouses te maken.
- Datawarehouses zijn niet heel geschikt voor Advanced Analytics.

Alleen relationele bronnen – Lange tijd beschikten we eigenlijk alleen over gegevens die bestonden in relationele databases van operationele databases. Het database-ontwerp en de kwaliteit van de gegevens voldeden vaak niet aan de eisen die Business Intelligence eraan stelt, maar met het feit dat de gegevens waren opgeslagen in een relationele database was niets mis. En dus werden datawarehouses ook gemaakt met relationele databases.
Daar waar we wel andere soorten gegevens hadden, woog de meerwaarde van de daarin verscholen informatie vaak niet op tegen de kosten die we maakten om de informatie uit die gegevens te halen. De signal-to-noise-ratio was te groot en/of de benodigde rekenkracht te duur.
Nu veel gegevens ontstaan die niet van nature gestructureerd zijn en in relationele databases zitten, is het minder logisch om een relationeel datawarehouse te bouwen. Omdat we de mogelijkheden hebben om uit alle gegevens zinvolle informatie te halen, willen we alle gegevens, op een voor Business Intelligence-doeleinden goede manier, langdurig opslaan. Voor machine learning-doeleinden heb je vaak meer aan de oorspronkelijke ruwe gegevens dan aan door ETL-processen aangepaste gegevens. Voor klassieke BI-doeleinden als het maken van rapporten en dashboards streven we voldoende flexibiliteit na. We moeten agile zijn.
Een Data Lake is het nieuwe datawarehouse. Het is een verzamelplek voor gegevens. Allerlei soorten gegevens, wat voor gegevens dan ook. Formaat, volume, herkomst, kwaliteit, het maakt allemaal niet uit. Je kunt het bewaren in je Data Lake.



Een **Data Lake** is een omgeving waar je ruwe gegevens verzamelt in hun oorspronkelijke formaat en schema. Meestal gaat het om tekst files en blob files.

Realtime datawarehousing – In klassieke datawarehousing-implementaties worden meestal elke nacht de nieuw ontstane gegevens van de afgelopen dag verwerkt in het datawarehouse. Alle rapporten, dashboards en analyses gebruiken dus de gegevens tot en met gisteren, maar hebben nieuwe gegevens die vandaag zijn ontstaan nog niet tot hun beschikking. Je kunt natuurlijk proberen om het datawarehouse tweemaal per dag te verversen of misschien zelfs elk uur. In de praktijk blijkt dat je bij een hoge frequentie van verversen steeds meer moeite moet doen om alles goed te laten werken. Dat betekent ook dat het steeds meer geld kost.
We zien, zeker met IoT-omgevingen, steeds meer gegevens die in realtime verwerkt moeten worden. Dat wordt streaming data genoemd. Relationele databases hebben veelal te veel overhead om dit goed te ondersteunen. Denk aan de V van Velocity. Een Data Lake, dat eigenlijk niets meer is dan bestanden op een opslagmedium, kan prima bestanden bevatten waarin door streaming-applicaties continu nieuwe gegevens worden geschreven.

Advanced Analytics – In hoofdstuk 9 leer je meer over analytics. Advanced Analytics gebruikt gegevens net anders dan Basic Analytics. Gegevens vooraf prepareren en in een datawarehouse zetten is vooral bedacht om basic analytics goed te ondersteunen. Het is echter zonde om veel moeite te steken in de gegevens prepareren als ze vervolgens gebruikt worden voor Advanced Analytics en de preparatie slecht blijkt te zijn voor dit doel. Het idee van een Data Lake is dat je ruwe, ongeprepareerde gegevens opslaat. De preparatie doe je pas als je een specifiek doel voor de gegevens hebt.

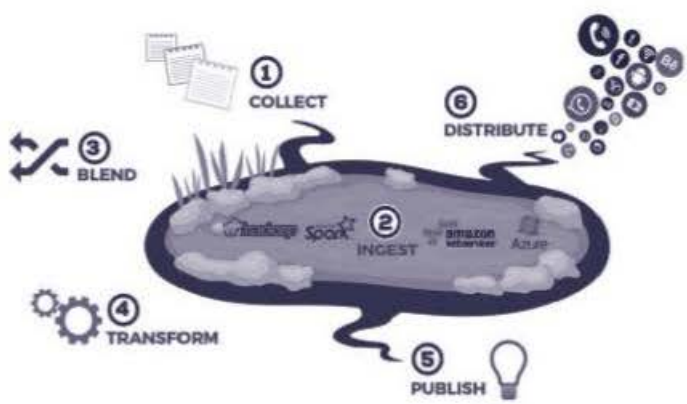
Nu we enkele beperkingen van relationele datawarehouses kennen, is het tijd om verder in te zoomen op het begrip en het nut van Data Lakes.

5.2.2 Wat is een Data Lake?

Het woord 'Lake' komt van een analogie met water. Stel, je hebt water en je krijgt constant nieuw water doordat het regent, door bronnen in de bodem en door rivieren die nieuw water aanvoeren. Je kunt dat water gewoon laten wegstromen, omdat je er niets mee kunt of niet weet wat je ermee kunt, of omdat je nu even geen behoefte hebt aan water, of omdat het te duur is om dat water op te slaan. Je kunt het water ook opvangen in een meer. Wellicht kun je, nu of later, geld verdienen met het water. Het water is hier een analogie voor digitale gegevens. We maken een meer vol digitale gegevens.
Wie wat bewaart, die heeft wat. Als je dorst krijgt, loop je naar het meer om wat te drinken. Misschien bedenk je wel dat je water in flessen kunt stoppen die je kunt verkopen. Er valt geld mee te verdienen. Als je huis in brand staat, bedenk je dat je water in emmers kunt doen om de brand te blussen. Als je stroom nodig hebt, laat je het uit je meer lopen om waterkrachtenergie op te wekken.

Wie weet wat je nog meer met water kunt. Wie weet wat je nog meer met data kunt!

De moraal is: alle gegevens hebben waarde. Wat je weggooit, ben je definitief kwijt. Dus gooi niets weg. Zelfs als je nu niet weet wat je met de gegevens kunt doen, is het zonde om ze weg te gooien. Met digitale foto's konden we een paar jaar geleden niet zoveel. Nu kunnen we door de vooruitgang in machine learning gezichtsherkenning en emotiedetectie loslaten op foto's en daarmee nuttige informatie boven water krijgen. Tenzij je al je foto's al hebt weggegooid natuurlijk...
Stel je voor dat je besluit om al het water dat je hebt in flessen te bewaren. Dat is ideaal als je dorst hebt. Het water komt in een handig formaat, je kunt meteen drinken. Maar stel nu dat er brand is en je met jouw in flessen bewaarde water de brand moet blussen. Het water stroomt zo langzaam uit de fles dat de brand waarschijnlijk nooit geblust gaat worden. Als het water nog in zijn oorspronkelijke formaat in het meer zou zitten, zou de brand blussen een stuk beter werken. Je moet het water, en in ons geval de gegevens, pas prepareren als je een specifiek gebruik voor ogen hebt. Niet eerder. Een datawarehouse maakt alles tot kolommen in tabellen, een Data Lake slaat ruwe gegevens op in hun oorspronkelijke formaat. Daarmee blijven alle mogelijke gebruiken van de gegevens nog mogelijk.
Een impliciete aanname die bij Data Lakes wordt gemaakt, is dat het opslaan van gegevens tegenwoordig niet meer duur is. Harde schijven zijn tegenwoordig groot in capaciteit en goedkoop. Ook de cloud speelt hier een rol. De cloud heeft een oneindige capaciteit, zolang je maar betaalt.
Goedkoop is niet gratis. De echte aanname is dat wat je ooit met de opgeslagen gegevens in je Data Lake gaat doen, meer waard is dan de kosten die je moet maken om het zolang te bewaren.



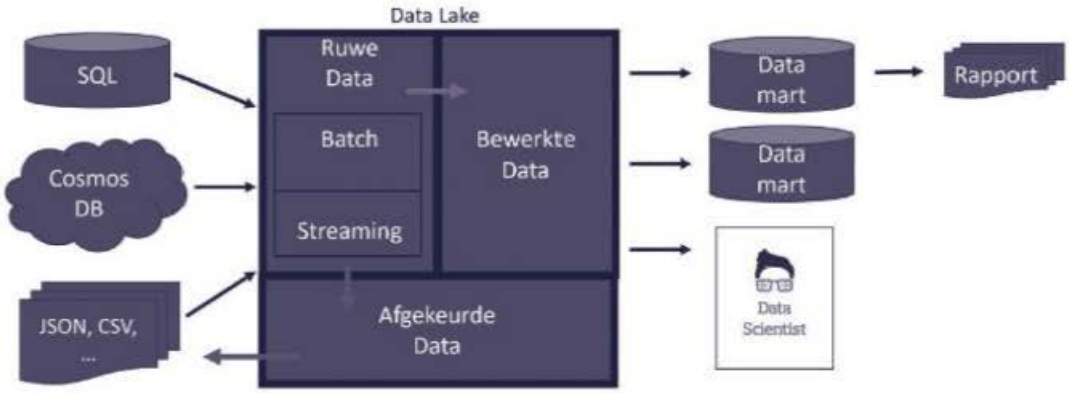
Figuur 5.1 Data Lake

Een Data Lake is dus niets anders dan een grote verzameling gegevens die nog niet bewerkt zijn voor een specifiek doel. Bij sommige bedrijven is Data Lake een concept dat zich niet een-op-een laat vertalen naar een product of een systeem. De verzameling van alle gegevens wordt het Data Lake genoemd. Daar hoort het datawarehouse waar veel gestructureerde gegevens worden verzameld en bewaard bij. Bij het Data Lake horen net zo goed de semi- en ongestructureerde gegevens. Die kunnen bewaard worden in noSQL-databases zoals document-databases. Gegevens kunnen ook worden opgeslagen als losse bestanden op een harde schijf, of als bestanden in bijvoorbeeld een Hadoop-cluster. Hadoop is een open source Big Data-platform. Het Data Lake wordt in bovenstaand geval gevormd door meerdere producten samen. De term Data Lake is meer een concept dan iets anders.
Vaak wordt een Data Lake geïmplementeerd als een verzameling files op een opslagmedium. Omdat een Data Lake over het algemeen heel veel gegevens moet kunnen bevatten (de V van Volume), zal dat opslagmedium niet 'gewoon' een harde schijf zijn. On-premise moet je denken aan een Hadoop-cluster of iets vergelijkbaars. Veelal worden Data Lakes in de cloud gemaakt. Steeds meer gegevens zijn cloud-born (ontstaan in de cloud) en de cloud is makkelijk en goed te schalen als er meer en meer gegevens in je Data Lake komen.
Alle gegevens opslaan in bestanden maakt je Data Lake ook flexibel. Uit alle soorten applicaties en databases kun je gegevens exporteren naar bestanden. Deze bestanden zijn makkelijk te kopiëren naar je Data Lake. De combinatie maakt dat je met elk databaseproduct dat je in gebruik hebt, nu of in de toekomst, de gegevens ervan kunt opnemen in je Data Lake. Bovendien maakt het niet uit of gegevens al in de cloud zitten of nog niet. Ook maakt het niet uit of je een cloud-based Data Lake bouwt. Bestanden vallen altijd te kopiëren naar het Data Lake.

Nu we weten wat een Data Lake is, is het tijd om te kijken hoe een Business Intelligence-architectuur met een Data Lake eruitziet. Daarna zullen we kort kijken hoe je zo'n Data Lake kunt opzetten.

5.2.3 *Architectuur met een Data Lake*

Een Business Intelligence-architectuur met een Data Lake is eigenlijk niet anders dan een Data Vault-architectuur. Dat is best logisch. Data Vault focust op de effectieve en flexibele manier van opslag en laat het gebruik over aan datamarts die later per use case gebouwd kunnen worden. Een Data Lake bevat ruwe gegevens en laat het gebruik over aan een later stadium. Figuur 5.2 laat de architectuur zien.



Figuur 5.2 Architectuur met een Data Lake

In figuur 5.2 zien we dat er verschillende soorten gegevens naar het Data Lake 'stromen'. Rechts van het Data Lake staan datamarts. Dit zijn dimensioneel gemodelleerde databases die de bron vormen voor rapporten en dashboards die de organisatie gebruikt. Datamarts zijn identiek aan zoals ze in een Data Vault-architectuur gebruikt worden. Elke datamart dient een specifiek functioneel doel en bevat alleen de gegevens die daarbij horen.
Je ziet in figuur 5.2 de data scientist apart genoemd worden. Voor machine learning en artificial intelligence is een dimensioneel gemodelleerde database vaak helemaal niet handig. Deze mensen kunnen we rechtstreeks toegang geven tot de bestanden in het Data Lake.

Op hoofdlijnen heeft het Data Lake van figuur 5.2 al wat structuur. Het Data Lake is opgedeeld in verschillende secties. Wat misschien het meeste opvalt is de sectie 'Bewerkte Data'. Tot nu toe heb ik immers gezegd dat het Data Lake ruwe onbewerkte gegevens moet bevatten.
Laten we linksboven beginnen. Nieuwe gegevens komen het Data Lake binnen in de sectie 'Ruwe Data'. Het gaat hier zowel om de eerder genoemde streaming data als de meer klassieke gegevens die we bijvoorbeeld elke nacht halen uit operationele systemen zoals ERP- en CRM-systemen. Deze systemen hebben nog altijd de problemen zoals benoemd in hoofdstuk 3: foutieve gegevens, incomplete gegevens en inconsistente gegevens komen altijd voor. Daarom worden de inkomende gegevens gevalideerd en gegevens met problemen worden verplaatst naar de sectie 'Afgekeurde Data'. De overige gegevens kunnen nu bijvoorbeeld gestandaardiseerd worden. Denk bijvoorbeeld aan datums overal in hetzelfde formaat zetten zodat gegevens later makkelijk te koppelen zijn. De sectie 'Bewerkte Data' bevat gevalideerde gegevens die bewerkt zijn volgens dit soort basale, altijd geldende, regels.

Merk op dat het grootste verschil tussen figuur 5.2 en figuur 4.12 uit het vorige hoofdstuk is dat er links meer soorten bronnen genoemd worden. Dit houdt meer rekening met het huidige data-landschap van bedrijven. Bovendien staat rechts in het plaatje de data scientist genoemd. Een Data Lake houdt ook meer

rekening met andere soorten data-analyses dan we in het verleden deden. Op hoofdlijnen zijn de plaatjes hetzelfde. Business Intelligence en met name de benodigde gegevensanalyses werken slecht op niet-geïntegreerde operationele systemen. Daarom brengen we alle gegevens samen op een plaats en op een manier die het beoogde gebruik mogelijk maakt. En dat alles in dienst van betere besluitvorming.

5.2.4 Structuur van een Data Lake

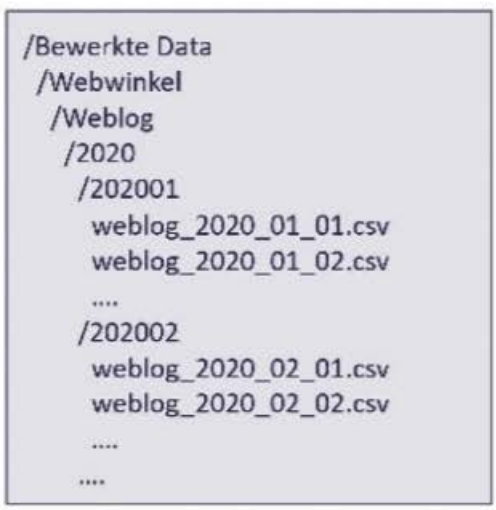
Met een gewone laptop of PC kan het al een uitdaging zijn om een goede folderstructuur te bedenken zodat je je bestanden makkelijk kunt terugvinden. Met een Data Lake is die uitdaging nog groter. Sommige bedrijven praten van een 'Data swamp' omdat ze dreigen te verdrinken in het moeras aan gegevens. Op hoofdlijnen kun je een Data Lake indelen in secties zoals in figuur 5.2. Per sectie zul je een verdere onderverdeling moeten maken. Figuur 5.3 laat een voorbeeld zien.



Figuur 5.3 Data Lake-folderstructuur

In figuur 5.3 zijn de secties 'Ruwe Data' en 'Bewerkte Data' folders in een hoofdfolder genaamd Data. De sectie 'Ruwe Data' is verder onderverdeeld in aparte folders per systeem dat ontsloten moet worden. Als er in de toekomst nieuwe systemen binnen de organisatie komen, kun je makkelijk een nieuwe folder maken en de gegevens daar opslaan. Het biedt de flexibiliteit om makkelijk nieuwe systemen op te nemen.
De sectie 'Bewerkte Data' is anders qua opzet. Dit is de sectie waar we mensen toegang geven om met de gegevens te werken. De beveiliging van de gegevens moet hier meegenomen worden. Het makkelijkste is om een folder te maken met gegevens die voor een specifieke doelgroep beschikbaar gemaakt kunnen worden. In de figuur worden de folders vernoemd naar projecten. Beter is waarschijnlijk functionele aandachtsgebieden. Alle sales-gerelateerde gegevens bij elkaar en alle gegevens omtrent personeelszaken in een aparte folder. Hoe we de gegevens gaan gebruiken moet nog geen rol spelen bij de indeling. Welke

gegevens bij elkaar horen, functioneel en vanuit beveiligingsperspectief bekeken, bepaalt welke folders je maakt.
Naast bovenstaande kun je twee soorten gegevens onderscheiden. Streaming data en transactionele data zoals orders en facturen, wil je vaak opslaan in incrementen. In andere woorden, je slaat alleen de nieuw ontstane gegevens op. Dimensiegegevens kunnen ook als snapshot opgeslagen worden, elke dag een dump van de situatie van die dag. Met name voor het incrementeel opslaan is de juiste folderstructuur van belang. Bekijk bijvoorbeeld figuur 5.4.



Figuur 5.4 Time slices

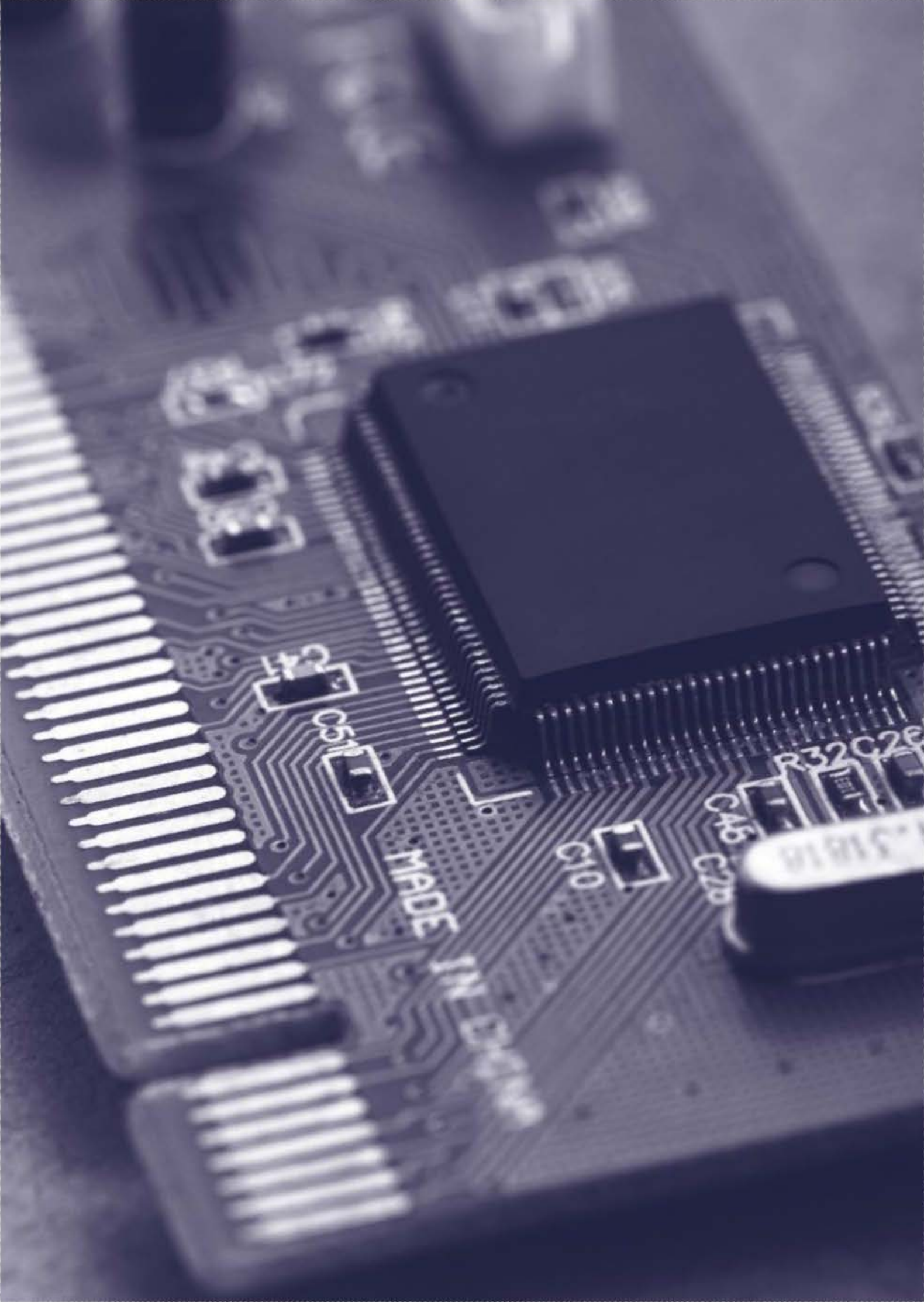
In de figuur zie je een folder per jaar. Daarbinnen zijn subfolders per maand gemaakt met daarin een bestand per dag. Daarmee kun je makkelijk alle gegevens van een jaar, alles van een maand of een individuele dag lezen. Veel applicaties staan toe dat je naar een folder verwijst en vervolgens alle gegevens uit die folder krijgt. Alle gegevens van een kwartaal inlezen is echter een stuk lastiger. Werk je binnen een organisatie met kwartalen, dan zou een extra folderlaag zeer aan te raden zijn.

5.3 Tot slot

Langzaam maar zeker gaan bedrijven over van datawarehousing naar de implementatie van Data Lakes. Voor veel organisaties in het midden- en kleinbedrijf (MKB) is dat (nog) niet zinvol. Onthoud (vooralsnog) de opmerking dat Big Data betekent dat je moet innoveren om je doel te bereiken omdat klassieke oplossingen met relationele databases niet meer tot het gewenste resultaat leiden. Voor veel organisaties zal gelden dat innoveren niet nodig is. '*Known and proven technology*', dus gebruiken wat bekend is en zich bewezen heeft, is vaak sneller en goedkoper dan voorop lopen alleen maar voor het voorop lopen.

Kijk bij het implementeren van Data Lakes ook naar de Business Intelligence-volwassenheid van een organisatie. Een bedrijf dat niet of nauwelijks data-gestuurd werkt, dus waar de kleine BI-cyclus nog niet werkt zoals beschreven in hoofdstuk 2, is nog niet toe aan Advanced Analytics en voorspellende algoritmes. Zet dat als stip op de horizon, maar begin bij het begin.
In dit hoofdstuk is een paar keer Advanced Analytics genoemd. Je leert daar meer over in hoofdstuk 9 dat gaat over het front-end en het gebruik van de gegevens. Eerst kijken we echter in het volgende hoofdstuk naar de fysieke aspecten die komen kijken bij het inrichten van datawarehouses en Data Lakes.

Op de website bij dit boek kun je terecht voor de oefenvragen bij hoofdstuk 5. 


C51
MADE IN CHINA
R32
C46
C10

Het fysieke datawarehouse

6

In het hoofdstuk 4 heb je geleerd over de logische tabelstructuur van een datawarehouse. De verschillen in benadering en structuur tussen een dimensioneel model, een genormaliseerd model en een Data Vault zijn besproken. In dit hoofdstuk ga je meer kijken naar de fysieke implementatie. Welke software kun je gebruiken? Wat voor soort hardware heb je nodig? Hoe zorg je ervoor dat de performance acceptabel zal zijn? Hoe houd je in de gaten wat de performance is? Allemaal zaken om over na te denken om een succesvolle implementatie van een datawarehouse te doen.

6.1 Inleiding

Tot nu toe hebben we het in dit boek heel generiek gehouden. Er is nog nergens gesproken over specifieke keuzes ten aanzien van hardware en software. In dit boek zal ook niet een keuze gemaakt worden. Toch is het goed om te kijken naar de mogelijkheden op dit gebied en naar enkele zaken die kunnen helpen bij het maken van een keuze.
Elke keuze die je maakt komt met voor- en nadelen. Alle hardware en software heeft zo zijn beperkingen. Hoewel alle producten anders zijn, zijn er ook veel overeenkomsten. Daarmee zijn er ook algemeen toepasbare manieren om bijvoorbeeld performance van query's te verbeteren. We zullen in paragraaf 6.4 van dit hoofdstuk enkele manieren bekijken. Om concrete voorbeelden te kunnen geven, maken we in dit boek gebruik van Microsoft SQL Server. Je kunt met de voorbeelden meedoen als je SQL Server installeert. Een gratis evaluatieeditie is te downloaden van de Microsoftsite (www.microsoft.com/sqlserver).
Vroeg of laat zul je tijdens een Business Intelligence-project een keuze moeten maken voor de te gebruiken hardware en software. Welk DBMS (Database Management Systeem) ga je gebruiken? Wordt het een op Microsoft SQL Server gebaseerd datawarehouse, moet het datawarehouse in Oracle gebouwd worden, kies je voor Teradata of wordt het een van de vele andere spelers op deze markt? Soms moet deze keuze nog echt gemaakt worden, vaak zul je geconfronteerd worden met een bestaande situatie. Niet altijd is de keuze voor hardware en software nog vrij. Het kan goed zijn dat er een eis is dat het datawarehouse gebruikmaakt van de bestaande, aanwezige hardware. Ook voor de software kan al lang een keuze zijn gemaakt, al was het maar omdat alle al bestaande databases op basis van hetzelfde DBMS zijn gemaakt.

Voordat je bovenstaande keuzes maakt moet je beslissen of je in de cloud gaat werken of niet. Hoe meer Big Data je implementatie bevat, hoe meer het voor de hand ligt om de cloud met zijn schaalbaarheid te gebruiken. Maar ook voor kleinere meer klassieke datawarehouse-implementaties is de cloud een steeds voor de hand liggender optie. Je moet dan uiteraard voor een cloud-provider kiezen. Bovendien moet je ook dan de keuze maken welke cloud services te gebruiken. Azure, Microsofts cloud, kent bijvoorbeeld meerdere soorten relationele databases en heeft speciale services gericht op Big Data. De keuze die je maakt kan grote gevolgen hebben voor de schaalbaarheid aan de ene kant en de kosten aan de andere kant.
Als je nog helemaal vrij bent in het maken van de keuzes zou Gartner (gartner.com) een startplek kunnen zijn om eens te kijken wat er zoal op de markt is. Zij publiceren elk jaar het datawarehouse magic quadrant. Figuur 6.1 toont het magic quadrant van 2021 voor Analytics en Business Intelligence. De producten van (bekende) leveranciers worden ingedeeld in de categorieën niche player, visionary, challenger en leader op basis van de compleetheid van het product, de visie rond het product en of ze daadwerkelijk hun beloftes waarmaken.



Figuur 6.1 Gartner Magic Quadrant for Analytics and Business Intelligence-platforms

Gartner stelt overigens op vele gebieden zo'n magic quadrant op. Zo is er bijvoorbeeld ook een over Data Integration, Data Science et cetera. Aangezien het datawarehouse maar een klein onderdeel is van de totale Business Intelligence-oplossing is het zeker interessant om ook de conclusies van andere relevante magic quadranten te bekijken. Houd bijvoorbeeld de BI-roadmap in de gaten zoals besproken in hoofdstuk 2 en zoek de relevante quadranten op passend bij de uitgesproken ambities in de roadmap.

6.2 Technisch ontwerp

Voordat je iets zinvols kunt zeggen over welke hardware en software je nodig hebt voor een succesvolle implementatie, moet je eerst meer weten over je datawarehouse. Hoe groot gaat het datawarehouse worden? Welke eisen worden er gesteld aan het datawarehouse in termen van beschikbaarheid, data latency en performance? Hoeveel gebruikers zijn er en hoe complex is de workload?
Een goed punt om mee te beginnen is het kwantificeren van het logische model dat je hebt overgehouden aan hoofdstuk 4. De tabelstructuur die een gevolg is van je ontwerp (sterschema of anderszins) wordt wel het functionele of soms het logische ontwerp genoemd. Van dit functionele ontwerp moet nu een technisch ontwerp worden gemaakt.



Het **functionele ontwerp** van een database is de gewenste tabelstructuur, alleen rekening houdend met de functionele eisen zoals die naar voren komen uit de informatieanalyse.

Het doel van het maken van een technisch ontwerp is om het functionele model aan te passen aan de (niet ideale) werkelijkheid. Er kan zich een tweetal problemen voordoen:

- de performance is niet toereikend;
- de structuur is niet werkbaar gezien beperkingen van het gebruikte DBMS.

Om iets zinvols te zeggen over deze mogelijke problemen, moet je weten hoe groot de verschillende tabellen gaan worden. De grootte van een tabel wordt bepaald door:

- het aantal records in een tabel;
- de gemiddelde recordlengte.

De gemiddelde recordlengte hangt uiteraard af van de inhoud van het datawarehouse (lengte van namen en omschrijvingen), maar ook van het datatype van de kolommen. In onderstaande paragrafen worden beide punten verder verklaard.



Het **technisch ontwerp** van een database past het functionele ontwerp zodanig aan dat de database in de praktijk goed functioneert.

6.2.1 Aantal records

Als eerste moet je een inschatting proberen te maken van het aantal records dat je per tabel verwacht. Voor sommige tabellen is dat gemakkelijk in te schatten, voor andere is het moeilijker. Een datumdimensie is bijvoorbeeld rechttoe rechtaan. Als het datawarehouse gegevens bevat vanaf 1 januari 2020 en je de datumdimensie vult met gegevens tot 2030, heb je tien jaar aan records. Nu is het nog een kwestie van de grain weten. Met een grain op dagniveau, krijg je een tabel met 3650 records. Niet echt spannend.
Het spannendst zijn waarschijnlijk de feitentabellen. Dit zijn namelijk over het algemeen de grote tabellen. Als je bijvoorbeeld gespreksduur van gesprekken naar een callcenter opslaat, en je krijgt gemiddeld duizend gesprekken per dag, hebben we het al over 365.000 records per jaar. Met tien jaar historie is dit meer dan 3,5 miljoen records. In een modern datawarehouse is dat nog een kleine tabel. Sommige feitentabellen bevatten miljarden records. Denk maar eens aan een datawarehouse van een landelijke supermarkt waar elk gescand product bij de kassa leidt tot een nieuwe rij in de feitentabel.
Het belangrijkste hier is dat je in orde van grootte probeert een inschatting te maken van het aantal records, zowel nu als in de toekomst. Als je ziet aankomen dat een tabel erg groot gaat worden, moet je vooraf nadenken over hoe je die omvang kunt beheren. Wellicht dat partitionering van de tabel zinvol is. Later in dit hoofdstuk zie je een voorbeeld van tabelpartitionering. Vooraf strategieën bedenken en implementeren om grote tabellen beheersbaar te houden is niet moeilijk. Achteraf dezelfde oplossing implementeren zou wel eens erg duur kunnen zijn. Beter voorkomen dan genezen, dus!



Sommige tabellen in datawarehouses worden zo groot dat ze onbeheersbaar worden. Als je dat vooraf ziet aankomen, kun je maatregelen treffen.

Je kunt een databasediagram uitbreiden met de bedachte aantallen. Een tabel wordt dan weergegeven zoals in figuur 6.2. Je ziet de naam van de tabel en de primaire sleutel. In het voorbeeld zie je de tabel dimDate met de sleutel DateKey. Tussen haakjes staat het datatype van de sleutel. Op de onderste regel zie je links de gemiddelde recordlengte in bytes en rechts het aantal te verwachten records. Vermenigvuldigd levert dit de grootte van de tabel in bytes op.



Figuur 6.2 Kwantificeren

6.2.2 Gemiddelde recordlengte

Naast het aantal te verwachten records is de lengte van de records belangrijk. Om dat te bepalen moet je drie zaken weten:
- Uit welke kolommen bestaat de tabel?
- Wat is het datatype van elke kolom?
- Wat is de gemiddelde lengte van bijvoorbeeld namen?

Uit het functionele model dat het resultaat is van wat je in hoofdstuk 4 hebt geleerd, kun je direct aflezen uit welke kolommen een tabel bestaat.
De volgende stap is te specificeren welk datatype de kolommen gaan krijgen. Daarvoor moet je weten welk DBMS je gaat gebruiken. Alle databases hebben vergelijkbare datatypes. Vergelijkbaar betekent echter niet hetzelfde.

Het **datatype** van een kolom bepaalt het soort gegevens dat in deze kolom opgeslagen kan worden, bijvoorbeeld een datum of een getal.

Grofweg heeft een database drie datatypes:
- numeriek;
- alfanumeriek;
- datum.

Dit lijstje kan worden aangevuld met grote datatypes. Hierbij moet je denken aan afbeeldingen, XML, grote tekstvelden en hele documenten.

Numeriek

Numerieke datatypes kunnen worden onderverdeeld in gehele en gebroken getallen. SQL Server kent bijvoorbeeld vier soorten gehele getallen, de tinyint, de smallint, de int en de bigint. Deze zijn respectievelijk 1, 2, 4 en 8 byte groot. Hoe groter het datatype, hoe groter de getallen die je kunt opslaan, maar ook hoe meer opslagcapaciteit en geheugen je nodig hebt. Een tinyint kan alleen positieve getallen opslaan met een maximum van 255. Dat is meer dan genoeg voor een kolom DagNummerVanWeek (1 t/m 7), DagNummervanMaand (1 t/m 31), maar al te klein voor DagNummerVanJaar (1 t/m 366). De andere integer datatypes kunnen zowel positieve als negatieve getallen opslaan met als

maximum 2 tot de macht (n-1) met n het aantal bytes maal 8. Voor de bigint is dat 9223372036854775807.



Hoe groter het datatype, hoe groter de getallen die je kunt opslaan, maar ook hoe meer opslagcapaciteit en geheugen je nodig hebt.

Gezien de hardware van tegenwoordig kun je je afvragen hoe belangrijk het is om je druk te maken over het verschil tussen 1 byte en 8 byte. Hoewel dat een terechte vraag is, moet je bedenken dat het om grote aantallen rijen gaat. Stel dat een tabel een miljard records bevat. Het verschil is dan opeens zeven miljard byte, ofwel 7 gigabyte. En we hebben het hier nog maar over één kolom! Los van de storage die dat kost, houdt het ook inefficiënt geheugengebruik in. Dat kan leiden tot meer IO (interactie van de database met de harde schijf), wat kan leiden tot slechte performance. Hoewel het argument aanzienlijk minder belangrijk is dan enkele jaren geleden, blijft het de moeite waard om naar te kijken.



Kies altijd het kleinste datatype dat gegarandeerd groot genoeg is voor wat je met een kolom gaat doen.

Naast gehele getallen zijn er gebroken getallen. Gebroken getallen kunnen vaak nog worden onderverdeeld in exacte datatypes en benaderde datatypes. Voor benaderde datatypes geldt over het algemeen dat ze een groter bereik hebben, dus dat ze grotere getallen kunnen bevatten. Daar staan afrondingsproblemen in berekeningen tegenover. Je kunt zelf aangeven hoe groot de precisie van deze datatypes moet zijn, dus hoeveel posities achter de komma je wilt bijhouden. Hier geldt natuurlijk ook weer dat hoe groter het bereik en hoe groter de precisie, hoe meer storage en geheugen je nodig hebt.
Afhankelijk van het gebruikte DBMS zijn er nog andere numerieke datatypes. SQL Server kent bijvoorbeeld money en smallmoney, die respectievelijk 8 en 4 byte geheugen gebruiken en een precisie kennen van vier posities achter de komma.

Alfanumeriek

Alfanumerieke velden kennen over het algemeen een variant met vaste lengte en een variant met variabele lengte. Voor namen is de laatste variant het meest geschikt. Voor een postcodekolom kun je beter een vaste lengte kiezen.
Een variabele lengte datatype gebruikt evenveel opslagcapaciteit en geheugen als dat er nodig is. Je geeft alleen op wat de maximale lengte kan zijn. Per DBMS kan deze maximumlengte verschillen.
Belangrijk bij alfanumerieke velden is nog de code page die je gaat gebruiken. De code page bepaalt welke karakters de database ondersteunt. Het is belangrijk om namen goed te kunnen opslaan, ook van mensen met een buitenlandse

afkomst waarvan de namen karakters bevatten die niet in het Nederlandse alfabet voorkomen. Om zo flexibel mogelijk te zijn is het verstandig om Unicode te gebruiken. De database gebruikt dan twee (of meer) bytes per karakter in plaats van één, maar kan wel alle mogelijke karakters opslaan. De functionele eisen en flexibiliteit lijken het hier in de praktijk wel te winnen van de storage- en performance-eisen.

Datum

Voor het opslaan van datums bestaat over het algemeen een apart datatype. In sommige systemen wordt een datum altijd gecombineerd met een tijd. Dit kan van belang zijn bij het schrijven van de juiste query's.

Gemiddelde lengte van namen

Nadat je van alle kolommen het datatype hebt bepaald, moet je nog de inhoud kennen. Het is niet genoeg om te weten dat een kolom ProductNaam het datatype `varchar(50)` heeft. Je moet ook weten uit hoeveel letters een productnaam gemiddeld bestaat.

Als je van elke kolom het datatype kent, en als je van alle kolommen met een variabele lengte de gemiddelde lengte kent, kun je de gemiddelde recordlengte uitrekenen. Maar waarom is dat belangrijk?

6.2.3 Page size

De meeste databases slaan records op pagina's op. Bij SQL Server is zo'n pagina altijd 8 kB groot. Na het aftrekken van de overhead blijft er 8060 byte vrije ruimte over. Twee vragen die boven komen drijven bij deze wetenschap zijn:

- Wat gebeurt er als de gemiddelde recordlengte groter is dan een pagina?
- Wat gebeurt er als het gemiddelde groter is dan een halve pagina?

Grote records

SQL Server slaat een record in principe op één pagina op. Je kunt niet twee vaste lengtekolommen opnemen in een tabel die beide bijvoorbeeld 5000 byte groot zijn. Alle vaste lengtekolommen samen moeten minder dan 8060 byte groot zijn. Een record kan wel groter worden dan 8060 byte, als dat maar komt door variabele kolommen. Alle gegevens die niet meer op de pagina passen, worden in een zogenaamd overflowgebied opgeslagen. Het ophalen van die gegevens is echter minder efficiënt dan het inlezen van een pagina. Je betaalt dus een prijs in termen van performance.

Bij dimensietabellen kan het bovenstaande een probleem worden. De theorie zegt dat je zoveel mogelijk beschrijvende kolommen moet opnemen in de dimensie. Daarmee loopt de gemiddelde recordlengte al snel op. Inefficiëntie en dus slechte performance liggen op de loer.

Met het toepassen van verticale partitionering of snowflakes kun je de problemen voorkomen. Je offert het ideale stermodel op ten behoeve van de

performance. In paragraaf 6.4 zie je een voorbeeld van het effect van verticale partitionering.



Het is belangrijk om in een vroeg stadium de gemiddelde recordlengte en page size te weten, zodat je een model kunt maken dat rekening houdt met beperkingen waardoor er later geen performanceproblemen zullen ontstaan.

Bij Oracle heb je een extra optie als records gemiddeld erg groot worden. Oracle slaat records op in blocks. Een block in Oracle komt overeen met een pagina in SQL Server. In Oracle kun je de grootte van een block zelf instellen. Je hoeft dus niet per se de tabelstructuur aan te passen, je kunt de database zo configureren dat hij goed aansluit bij de tabelstructuur.



Je moet altijd kijken hoe het functionele ontwerp aansluit bij de eventuele beperkingen van het DBMS en aanpassingen doen waar dat nodig is.

Paginavulling

Stel je eens de volgende heel extreme situatie voor. Alle records zijn precies 1 byte groter dan een halve pagina. Het eerste record dat je opslaat komt op de eerste pagina te staan. Het tweede record past net niet meer op de eerste pagina en komt dus op een tweede pagina. En zo verder voor alle volgende records. Grofweg heb je een gigabyte aan storage nodig om 500 MB aan gegevens op te slaan. Ook het werkgeheugen van de computer zal op gelijke inefficiënte manier gebruikt worden. Los van storagekosten en het extra geheugen dat de server nodig heeft, zal dit resulteren in slechte performance.
Uiteraard zal het in de praktijk niet zo extreem zijn als hierboven beschreven. Het effect treedt echter wel degelijk op, inclusief de bijbehorende negatieve impact op de performance.

Samenvatting

Om zicht te krijgen op eventueel te verwachten performanceproblemen wil je weten hoe groot tabellen zijn. Om dat te berekenen moet je het aantal records kennen en de gemiddelde recordlengte. Voor de gemiddelde recordlengte moet je het datatype van alle kolommen weten en het gemiddeld aantal karakters dat datatypes met variabele lengte gaan bevatten.
Met deze informatie kun je het functionele ontwerp van het datawarehouse vertalen naar een technisch ontwerp. Het doel is om problemen met performance en beheersbaarheid van het datawarehouse voor te zijn.

6.2.4 Grootte van de database

Met de informatie uit de vorige paragrafen kun je de grootte van het datawarehouse berekenen. Er is echter nog een aantal factoren dat een impact heeft op de totale omvang van de database. Je moet dan vooral denken aan:

- compressie van de gegevens;
- indexen.

Compressie

De meeste DMBS'en kennen tegenwoordig compressie van gegevens. Populaire programma's zoals Winzip en 7zip maken bestanden op je harde schijf kleiner. Dat is handig als je schijfruimte tekortkomt of bij het uitwisselen van gegevens (downloaden van het internet, cloud storage zoals Dropbox, gebruik van kleine usb-flashdrives). Het is ook erg handig bij grote tabellen. Je bereikt namelijk twee dingen met compressie:

- Je hebt minder opslagcapaciteit nodig.
- Je krijgt (meestal) betere performance.

Minder opslagcapaciteit is gezien de kosten van opslag misschien niet heel erg spannend. Maar van kleinere databases kunnen ook sneller back-ups gemaakt worden. Daarmee wordt ook op de opslag van back-ups bespaard. Bovendien is er 's nachts meer tijd voor andere zaken (ETL, processen van kubussen) als de back-up korter duurt. Daarnaast kunnen kleine back-ups sneller gerestored worden. Restore-tijden kunnen belangrijk zijn voor afgesproken SLA's (Service Level Agreement). Een kleinere database heeft dus veel voordelen.
Vaak denken mensen dat compressie weliswaar goed is voor de benodigde opslagcapaciteit, maar dat het ten koste gaat van de performance. Performanceverbetering kan echter een belangrijke reden zijn om compressie toe te passen. Bij query's waarvoor gegevens van de harde schijf gehaald moeten worden, is de performance van die schijf vaak de beperkende factor. Schijven zijn over het algemeen veel trager dan geheugen. Door het toepassen van compressie hoeven er minder gegevens direct van de schijf ingelezen te worden. Daarmee gaan query's beter performen.
In paragraaf 6.4 van dit hoofdstuk kun je een voorbeeld van compressie zien.

Indexen

Naast compressie hebben vooral indexen impact op de grootte van de database. Een index neemt altijd extra ruimte in beslag, de database wordt dus groter. Een index maakt het als het goed is makkelijker om bij de juiste gegevens te komen. Dat zorgt voor een betere performance van query's.
In paragraaf 3.2.2 heb je kunnen lezen over indexen. Voor het inschatten van de grootte van de database is het van belang om te weten welke indexen je gaat maken. Welke indexen nuttig zijn hangt volledig af van de query's die uitgevoerd gaan worden. In zijn algemeenheid kan een datawarehouse na het maken van indexen tweemaal zo groot worden.

In paragraaf 6.4 van dit hoofdstuk kun je een voorbeeld van indexing zien.

6.2.5 Overige factoren

Nu je enig zicht hebt op de grootte van de tabellen en de database als geheel, kun je gaan kijken naar de te verwachten performance van de database en de hardware die je daarvoor nodig hebt. Voordat we iets over hardware zeggen zullen we stilstaan bij:

- de te verwachten workload;
- het aantal gebruikers;
- beschikbaarheidseisen.

Workload

Om iets zinvols te kunnen zeggen over de performance moet je de workload kennen: welke query's zullen er uitgevoerd worden? Als je een goede informatieanalyse hebt gedaan, weet je dat. Welke rapporten moeten er gemaakt worden op het datawarehouse? Met welke frequentie en door hoeveel gebruikers wordt een rapport opgevraagd? Welke informatie staat op het rapport en tot welke query's leidt dat?

Het kan zijn dat alle rapporten gebaseerd zijn op een kubus. In dat geval is het processen van de kubus de enige workload op het relationele datawarehouse (datamart). Het andere uiterste is dat alle rapporten rechtstreeks hun gegevens door middel van complexe query's uit het relationele datawarehouse halen.

Om een inschatting te maken van de workload kun je query's indelen in simpele, gemiddelde en complexe query's:

- Simpele query's zijn gewone rechttoe rechtaan SELECT-statements, zonder moeilijke functies, berekeningen of constructies.
- Gemiddelde query's worden met een grote frequentie uitgevoerd (veelgebruikte rapporten) en bevatten een aantal joins en aggregaties.
- Moeilijke query's bevatten complexe logica. Soms moet een tabel vaker benaderd worden om de gewenste resultaten te berekenen. Een grote tabel benaderen zal altijd spannend zijn vanuit een performanceperspectief. Als die tabel dan ook nog enkele keren benaderd moet worden om tot de juiste resultaten te komen, wordt de performance een uitdaging.

Aantal gebruikers

Het aantal zogenaamde concurrent gebruikers is medebepalend voor de belasting op je systeem. Concurrent betekent gelijktijdig. Hoeveel gebruikers zullen er tegelijkertijd gebruikmaken van het datawarehouse? Als alle query's heel eenvoudig zijn, maar er worden er veel tegelijkertijd uitgevoerd, zal het systeem veel intern geheugen nodig hebben, zeker als de datasets die uit die query's komen groot zijn. Met maar enkele gebruikers die veel complexe query's uitvoeren is CPU-kracht waarschijnlijk belangrijker.

Beschikbaarheid

Ten slotte moet je nagaan welke eisen een organisatie stelt aan de beschikbaarheid van het datawarehouse. Sommige datawarehouses zijn lokaal en worden alleen gebruikt tijdens lokale kantoortijden. Andere datawarehouses zijn corporate datawarehouses met gebruikers van vestigingen over de hele wereld. Er is dan altijd wel ergens een kantoor open en dus wordt het datawarehouse 24 uur per dag gebruikt. Dit laatste scenario maakt bijvoorbeeld het laden van het datawarehouse een stuk lastiger.

Naast deze 'gewone' beschikbaarheid kunnen er nog eisen zijn met betrekking tot downtime. De servers zullen moeten worden 'gepatcht'. Service packs moeten worden geïnstalleerd en dat gaat vaak samen met het herstarten van de machine. Kan dat wel in een 24/7-omgeving?

Los van de geplande downtime voor onderhoud kan er altijd sprake zijn van ongeplande downtime. Wat gebeurt er als een computer uitvalt? Wat gebeurt er als de storage-oplossing uitvalt? Wat gebeurt er als een heel datacenter uitvalt? In termen van beschikbaarheid (high availability) zijn er veel mogelijkheden. Het is voornamelijk een kwestie van het afwegen van de risico's tegen de kosten. Wat is het risico dat er iets fout gaat, wat zijn de kosten als dat daadwerkelijk gebeurt en wat zijn de kosten om ons tegen dit risico te beschermen?

6.2.6 Cloud

Of een datawarehouse/datamart in de cloud gemaakt wordt of niet maakt voor het bovenstaande over het algemeen niet uit. Er zijn wel wat nuanceverschillen. In de cloud is het vaak minder erg als een database sneller is gegroeid dan verwacht. Dat maakt plannen makkelijker, omdat je makkelijker kunt opschalen op het moment dat dat nodig is.

Aan de andere kant geldt dat je in de cloud in veel gevallen betaalt voor wat je gebruikt. Een inefficiënt ontworpen database verbruikt meer resources dan een efficiënt ontworpen database. Dat maakt het direct duurder of soms indirect omdat je een grotere service tier nodig hebt.

Bij Microsoft in de cloud kun je grofweg kiezen tussen een SQL database of een Synapse SQL Pool als je een datamart gaat maken. Synapse SQL Pool is een service speciaal gebouwd voor hele grote datamarts. Een Synapse SQL Pool is daarom wel veel duurder dan een 'gewone' SQL Database. Je kunt niet makkelijk switchen tussen beide dus je zult vooraf moeten kiezen met welke service je de datamart gaat bouwen. Om een weloverwogen keuze te maken, moet je weten hoe groot je tabellen gaan zijn en welke performanceverwachtingen en -eisen er bestaan.

Voor zowel cloud als on-premise implementaties is het dus noodzakelijk om vooraf een inschatting te maken van de grootte en het gebruik van je datawarehouse dan wel datamart.

Samenvatting

De moraal van dit hoofdstuk en van deze paragraaf is: weet vooraf wat de organisatie van het datawarehouse verwacht. Bekijk alle factoren die van invloed zijn op deze verwachtingen. Stem het ontwerp van het datawarehouse en de verwachtingen van de organisatie op elkaar af.

6.2.7 Grootte van je oplossing

Uiteraard is het heel moeilijk om uitspraken te doen over hoe groot en hoe complex een Business Intelligence-oplossing is. Met wat tot nu toe besproken is, zou een onderverdeling er als volgt uit kunnen zien:

	Klein	Middelgroot	Groot
Volume gegevens	<= 1 TB	1 t/m 10 TB	> 10 TB
Query complexiteit	>= 50% eenvoudig +- 30% gemiddeld < 10% complex	+- 50% eenvoudig 30 - 35% gemiddeld 10 - 15% complex	30 - 35% eenvoudig +- 40% gemiddeld 20 - 25% complex
Aantal gebruikers	<= 100 gebruikers, 10-20 concurrent	<= 1000 gebruikers, 100-200 concurrent	>1000 concurrent gebruikers
Beschikbaarheid	Kantooruren	1 uur per nacht down	24 x 7 beschikbaar

Figuur 6.3 Datawarehouse-classificering

Veel leveranciers van zowel hardware als software kunnen op basis van dit soort classificeringen en ervaringen met andere klanten aanbevelingen doen. Het is altijd goed om te kijken naar ervaringen van anderen met overeenkomstige projecten. Al was het maar om niet het wiel opnieuw te hoeven uitvinden. Microsoft kent bijvoorbeeld de Fast Track Data Warehouse Architectures. Hierin vind je aanbevelingen voor hard- en software gebaseerd op jouw classificering en Microsofts ervaringen met eerdere projecten.

6.2.8 Data Lake

Bij een Data Lake-implementatie geldt al het bovenstaande voor de datamarts die je bouwt om de gegevens uit het Data Lake te ontsluiten naar rapporten en dashboards.

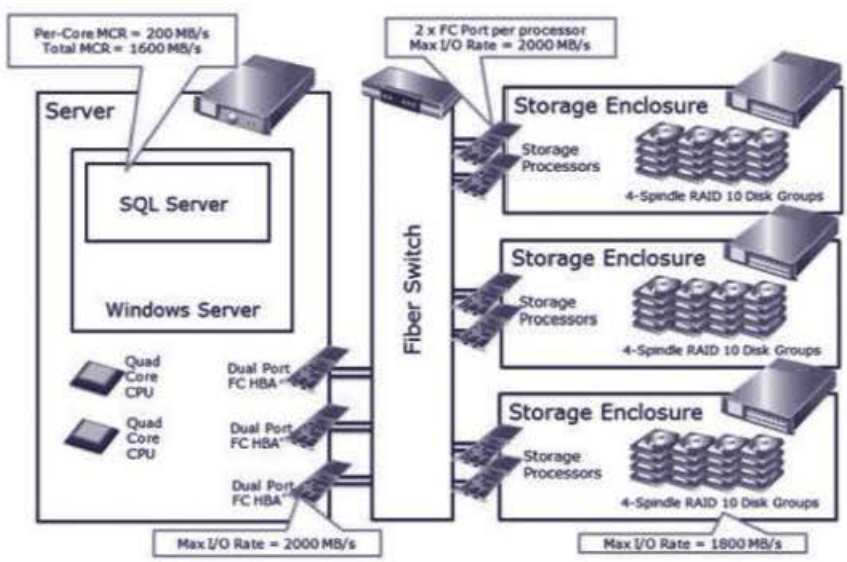
Voor het Data Lake zelf gelden overeenkomstige argumenten. Omdat het Data Lake bestaat uit bestanden, is het verhaal over datatypes niet van belang. Uitrekenen van een gemiddelde recordlengte is ook niet zo zinvol. Wel is het interessant om te weten hoeveel MB, GB of TB aan nieuwe gegevens per dag opgeslagen moeten worden. Veel Data Lake-implementaties zijn in de cloud. Daar betaal je over het algemeen per MB per maand voor de opslag van je gegevens.

In het vorige hoofdstuk heb je al geleerd dat de folderstructuur die je kiest voor je Data Lake bepalend kan zijn voor welke gegevens makkelijk op te vragen zijn. Daarnaast heeft het bestandsformaat nog veel invloed. Veelal worden zogenoemde Parquet files gebruikt. Dit zijn bestanden die gegevens gecomprimeerd opslaan en op een slimmere manier dan simpele CSV-bestanden. Compressie van gegevens is om dezelfde twee redenen als bij databases interessant: kostenbesparing en performanceverbetering. Slimmere opslagstructuren zullen de performance van het Data Lake verbeteren.

Nu je enigszins een idee hebt van de grootte en complexiteit van het datawarehouse, gaan we in de volgende paragraaf kort in op hardware. In paragraaf 6.4 gaan we kijken hoe verschillende features van producten invloed hebben op de performance van het datawarehouse.

6.3 Hardware en software

Het gaat voor dit boek te ver uitgebreid in te gaan op benodigde en beschikbare hardware. In algemene termen zullen we in deze paragraaf de specificaties bezien die aan de hardware gesteld moeten worden.
Alle specificaties die in de vorige paragraaf zijn genoemd, moeten leiden tot inzicht in de benodigde hardware en de te kiezen architectuur. In figuur 6.4 zie je een overzicht van de hardwarecomponenten die een rol spelen. Het plaatje is afkomstig van de zojuist genoemde Fast Track Data Warehouse van Microsoft, vandaar dat je links in het plaatje SQL Server en Windows Server ziet staan. Het idee is echter ook geldig voor andere systemen.



Figuur 6.4 Hardware-architecture

Bij de Fast Track-architecturen wordt ervan uitgegaan dat alle componenten vergelijkbare specificaties moeten opleveren voor wat betreft de hoeveelheid

gegevens die per seconde verwerkt kunnen worden. Als je bijvoorbeeld met een nieuw schijvenkabinet tweemaal zoveel bytes per seconde kunt lezen als met de oude harde schijven, maar de CPU of de netwerkkaarten kunnen niet meer data verwerken, heb je niets aan die verbeterde specificaties. Elke component kan de bottleneck zijn en de traagste zal dat zijn.
In figuur 6.4 is het systeem redelijk in balans. Het bestaat uit drie aparte storage enclosures in een SAN (Storage Area Network) met fiber channel tussen het SAN en de server. Bij elke component staat de maximale datacapaciteit. Deze getallen moeten in orde van grootte hetzelfde zijn. Anders is er sprake van een bottleneck en overcapaciteit bij de andere componenten.
Een goede plek om te starten met het doorrekenen van een systeem is het berekenen van de capaciteit van de CPU. De overige getallen worden over het algemeen wel gepubliceerd door de verschillende hardwareleveranciers. Er zijn veel tools op de markt (bijvoorbeeld SQLIO voor SQL Server) die helpen om de juiste getallen boven water te krijgen voor een specifiek systeem.

6.3.1 CPU-capaciteit

De Maximum Consumption Rate (MCR) is de maximale hoeveelheid gegevens die door één processorcore kan worden verwerkt. Met andere woorden, hoeveel MB per seconde kunnen maximaal verwerkt worden door één core? De MCR wordt bepaald door een combinatie van het soort processor, het moederbord van de server waarop die processor zich bevindt en het DBMS, of SQL Server in het geval van figuur 6.4. Doordat het gaat over de specifieke combinatie van deze drie componenten, zul je de MCR altijd zelf moeten meten op de beoogde hardware met de juiste software.



De **Maximum Consumption Rate (MCR)** is de maximale hoeveelheid data die het DBMS per seconde per processorcore kan verwerken.

Je kunt de MCR meten door een query uit te voeren en te kijken hoeveel data de query ophaalt en wat de query-responstijd was. Belangrijk bij het meten van de MCR is dat de query uitgevoerd wordt door maximaal één core. Bovendien moeten de opgevraagde gegevens uit het interne geheugen komen, anders meet je wellicht de performance van je netwerk of van je storage.

Nadat je de MCR hebt bepaald, kun je bepalen hoeveel cores je nodig hebt. Je moet daarvoor het volgende weten:
- Hoeveel gegevens (in MB) haalt een gemiddelde query op?
- Hoeveel gebruikers moet je tegelijkertijd ondersteunen (concurrent users)?
- Wat is de vereiste responstijd voor de query?

Met deze gegevens en onderstaande formule kun je uitrekenen hoeveel cores er nodig zijn om aan de performance-eisen te kunnen voldoen.

```
[Aantal cores] = ((grootte van resultset in MB / MCR) x aantal
concurrent gebruikers) / responstijd in sec.
```

6.3.2 Overige hardwarespecificaties

Naast de CPU moet je nadenken over de volgende vragen:
- Hoeveel geheugen heeft de datawarehouseserver nodig?
- Welke netwerkbandbreedte is nodig?
- Hoeveel opslagcapaciteit is er nodig en welke lees- en schrijfsnelheid is er nodig?

Geheugen

De benodigde hoeveelheid geheugen is moeilijk vast te stellen. Daar bestaat niet een mooie formule voor zoals hierboven beschreven voor de MCR. Gezien het feit dat gegevens ophalen uit het geheugen altijd veel sneller is dan gegevens ophalen van disks, zouden bij voorkeur alle gegevens in het geheugen moeten zitten. Er zijn ook steeds meer producten op de markt die daarop zijn gebaseerd. Het op één na beste is om genoeg geheugen te hebben zodat de actieve dataset in het geheugen past. Het actieve deel van de gegevens is dat deel dat regelmatig geraadpleegd wordt. Stel dat 90 procent van de query's data ophaalt die betrekking heeft op het lopende jaar. Dan is dat jaar onderdeel van de actieve dataset. Eerdere jaargangen zijn daar geen onderdeel van.



De **actieve dataset** zijn de gegevens die regelmatig geraadpleegd worden en zich daarmee bij voorkeur in het geheugen van de server moeten bevinden.

Uit de informatieanalyse moet naar voren zijn gekomen welke vragen er regelmatig beantwoord moeten worden. Daaruit volgt de actieve dataset. Je moet er nog wel voor zorgen dat ook precies de gevraagde gegevens van de harde schijven gelezen kunnen worden. Zonder indexen zal voor elk record dat opgevraagd wordt, de hele tabel ingelezen moeten worden. In dat laatste geval behoort alsnog de hele tabel tot de actieve dataset.

Er bestaat ook een vuistregel voor het bepalen van de actieve dataset. Uit onderzoek van Microsoft komt naar voren dat in een gemiddeld datawarehouse 20 procent van de opgeslagen gegevens behoort tot de actieve dataset.

Naast deze vuistregel zijn er nog ervaringscijfers op het gebied van hardware. Soms hoor je getallen als 4 GB per core. Of dat genoeg is moet de praktijk uitwijzen.

In volgorde van wenselijkheid bepaal je met onderstaande manieren de hoeveelheid benodigd geheugen:
1. gebruik zoveel geheugen dat alle gegevens in het geheugen passen. Anders,
2. gebruik zoveel geheugen dat de actieve dataset in het geheugen past. Anders,
3. zorg dat 20 procent van de gegevens in het geheugen past. Anders,
4. zorg voor minimaal 4 GB per core.

Netwerk

Het resultaat van elke query moet over het netwerk naar de gebruikersapplicatie gebracht worden. Als de resultaten groot zijn en/of als er heel veel query's worden afgevuurd, zal dat veel netwerkverkeer met zich meebrengen.
Uit de informatieanalyse komt naar voren welke rapporten hoe vaak gebruikt worden. In combinatie met de kennis hoe groot de dataset is die een rapport ophaalt, geeft dit informatie over de benodigde bandbreedte.

Opslag

Over opslag zou een heel boek volgeschreven kunnen worden (sterker nog, daar zijn hele boeken over volgeschreven). De open deur hier is dat je genoeg storage nodig hebt voor de hoeveelheid gegevens die je gaat opslaan. Vandaar een groot deel van de exercitie van paragraaf 6.2. Afhankelijk van het systeem dat je kiest, komt daar nog extra benodigde schijfruimte bij. SQL Server zal bijvoorbeeld bij grote query's gebruikmaken van tempdb, een systeemdatabase om tussenresultaten op te slaan. Ook tempdb heeft voldoende ruimte nodig.
Naast de hoeveelheid opslagcapaciteit is ook het soort opslagmedium belangrijk. Vanuit een performance-oogpunt is snelheid natuurlijk van belang. Wellicht dat solid state disks (SSD) een optie zijn. Deze disks zijn het duurste maar ook de snelste die op dit moment op de markt zijn. SSD's zijn de enige permanente opslag die qua performance in de buurt komen van memory.
SAN's (Storage Area Networks) zijn prima geschikt voor datawarehousing. Voornamelijk het beheer, en het laten meegroeien van de opslagcapaciteit met het datawarehouse, zijn pluspunten.

6.3.3 Appliances

Deze paragraaf gaat er tot nu toe min of meer van uit dat er een server is met daarop een DBMS. Vaak is dat wat we in de praktijk doen, maar wel vooral bij kleine en middelgrote datawarehouses zoals beschreven in de tabel in paragraaf 6.2.7. Voor heel grote datawarehouses is het interessant om te kijken naar datawarehouse-appliances.
Een appliance is hardware en software ineen. Microsoft verkoopt APS (Analytics Platform System, voorheen PDW, wat staat voor Parallel Data Warehouse), in samenwerking met DELL en HP. SQL Server komt als het ware ingebakken in de hardware, volledig en optimaal geconfigureerd voor grote datawarehouses.

Het functioneel ontwerp van het datawarehouse moet je uiteraard nog zelf maken. Je hoeft je niet druk te maken over zaken als file placement, RAID, parallellisme en andere configuraties. Dat soort zaken wordt geregeld door de appliance.

Een **datawarehouse appliance** is een geïntegreerde set van servers, opslagmedia, operating system, DBMS en andere software speciaal geïnstalleerd en geoptimaliseerd voor datawarehousing.

Uiteraard hebben ook de andere grote leveranciers vergelijkbare producten. Oracle heeft Exadata, IBM Netezza, en Teradata noemt het gewoon Teradata. Overal is het doel hetzelfde: voorgeconfigureerde hardware zodat je alleen nog maar met het functionele deel aan de slag hoeft. Met een beetje geluk is een hoofdstuk als dit in de toekomst helemaal overbodig.
Appliances hebben veel te bieden, maar zijn zeker niet goedkoop. Tussen zelf een gewone server helemaal configureren en inrichten voor het datawarehouse en een appliance zit nog wel een tussenweg, referenced architectures. Dit zijn specificaties opgesteld door hardwareleveranciers op basis van opgedane ervaringen bij klanten. Op basis van je eigen inschatting van grootte en de eisen ten aanzien van performance en beschikbaarheid, kunnen standaardoplossingen geboden worden.

Referenced architectures zijn vooraf gedefinieerde specificaties van hardware opgesteld met ervaring van vergelijkbare datawarehouseprojecten uit het verleden.

6.3.4 *Cloud-overwegingen*

Ook nu geldt weer dat er met de introductie van de cloud niet iets wezenlijks verandert. Je moet bij een cloud-implementatie net zo goed weten welke resources jouw workload nodig heeft om met de gewenste performance te draaien. Bij IAAS (Infrastructure as a Service)-oplossingen moet je een virtuele machine met een DBMS configureren. Dezelfde argumenten als tot nu toe genoemd zijn van toepassing. Je hebt alleen minder directe controle over de hardware. Over het algemeen kun je wel kiezen voor virtuele machines die relatief veel geheugen hebben ten opzichte van de CPU-capaciteit. Je kunt zelf het soort disks dat je wilt gebruiken kiezen waarbij meer performance neer zal komen op meer kosten.
Bij PAAS (Platform as a Service) heb je geen virtuele machine meer, maar slechts een database. Het service level dat je afneemt bepaalt hoe duur de database is en wat je daar in termen van capaciteit voor terugkrijgt. Over het algemeen is dit service level makkelijk aan te passen, maar dat is niet altijd het geval.

6.4 Performance features

Tot nu toe is in dit hoofdstuk vooral gekeken naar de hardware-aspecten van het inrichten van een datawarehouse. In algemene termen is gekeken naar benodigde hardware, voor een groot deel gebaseerd op hoe groot het datawarehouse wordt. Uiteraard is de workload, welke en wat voor query's en de drukte – hoeveel gebruikers – belangrijk. Ook de gekozen software, het DBMS, kan bijdragen aan een succesvolle implementatie. De meeste DBMS'en hebben overeenkomstige features.
Deze paragraaf is bedoeld om een meer concreet beeld te krijgen van wat een RDBMS kan. De voorbeelden zijn specifiek voor het product SQL Server van Microsoft en dus niet generiek. Daarom is deze paragraaf geen onderdeel van het gedrukte boek, maar kun je hem online lezen via www.boomstudent.nl.

De features waar we naar gaan kijken zijn:
- indexen;
- columnstore index;
- compressie;
- aggregatietabellen;
- partitionering.

6.5 Tot slot

In hoofdstuk 4 heb je geleerd welke soort tabelstructuren je kunt toepassen voor een datawarehouse. De daar geleerde theorie is onafhankelijk van het DBMS dat je gebruikt. In de praktijk is het noodzakelijk om het functionele model aan te passen aan de gebruikte hard- en software en aan de te verwachten workload. In dit hoofdstuk heb je geleerd welke extra aspecten van belang zijn om over na te denken voordat je een datawarehouse gaat implementeren.
Los van de hardware levert het DBMS features op die kunnen helpen om de juiste performance te halen. Voor het succesvol inzetten van deze features moet je de workload kennen. Je past het functionele databasedesign uit hoofdstuk 4 aan op het daadwerkelijke gebruik zoals dat uit de informatieanalyse is gekomen. Uiteraard zijn de beschikbare features afhankelijk van het gebruikte DBMS.

Op de website bij dit boek kun je terecht voor de oefenvragen bij hoofdstuk 6.

